Contents

1 Basis 1
- 1.1 Default Code . . . 1
- 1.2 Some Useful Primes . . . 1
- 1.3 偽隨機 & MT19937 . . . 1
- 1.4 對拍 code . . . 1
- 1.5 快讀快寫 . . . 1
- 1.6 怪怪的東西 . . . 1

2 Data Structures 1
- 2.1 Disjoint Set (併查集) . . . 1
- 2.2 Binary Indexed Tree . . . 2
- 2.3 二維 BIT . . . 2
- 2.4 Segment Tree . . . 2
- 2.5 Treap . . . 2
- 2.6 平板電腦 . . . 3

3 Mathematics 3
- 3.1 公式們 . . . 3
- 3.2 階乘 & 模逆元 . . . 3
- 3.3 GCD & LCM . . . 3
- 3.4 EXGCD . . . 3
- 3.5 Fast Power . . . 3
- 3.6 矩陣乘法 . . . 3
- 3.7 多項式 and 操作 . . . 4
- 3.8 FFT . . . 4
- 3.9 NTT . . . 5
- 3.10 質數表 . . . 5
- 3.11 歐拉函數 . . . 5
- 3.12 Miller-Rabin Algorithm . . . 5
- 3.13 Pollard's Rho Algorithm . . 6
- 3.14 高斯消去法 . . . 6
- 3.15 約瑟夫問題 . . . 6
- 3.16 莫比烏斯函數 . . . 6
- 3.17 離散對數 . . . 6
- 3.18 離散根號 . . . 6
- 3.19 XOR Basis . . . 7

4 Graph 7
- 4.1 建圖方法 . . . 7
- 4.2 Topological Sort . . . 7
- 4.3 Tarjan's Algorithm for BCC . 7
- 4.4 Bellmen-Ford Algorithm . . 7
- 4.5 Dijkstra's Algorithm 7
- 4.6 Dinic's Algorithm for Maximum Flow . . . 8
- 4.7 Dominator Tree . . . 8
- 4.8 Edmonds Blossom Algorithm 9
- 4.9 Floyd-Warshall Algorithm . 10
- 4.10 Maximum Clique . . . 10
- 4.11 Kosaraju's Algorithm for SCC 10
- 4.12 Kruskal's Algorithm for MST 11
- 4.13 Kuhn-Munkre Algorithm . . 11
- 4.14 Max-flow-min-cost . . . 11
- 4.15 Kuhn's Algorithm . . . 12
- 4.16 Flow Methods . . . 12

5 Tree Algorithms 12
- 5.1 時間戳記 & 倍增表 (初始化) . 12
- 5.2 LCA . . . 12
- 5.3 Eular Tour for LCA . . . 12
- 5.4 樹上差分 . . . 12
- 5.5 DSU on Tree . . . 13
- 5.6 樹鏈剖分 . . . 13
- 5.7 虛樹 . . . 13
- 5.8 樹重心 . . . 14
- 5.9 重心剖分 . . . 14
- 5.10 Tree Hash . . . 14

6 String Algorithms 14
- 6.1 Suffix Array (Binary Lifting) 14
- 6.2 Suffix Array (SAIS) . . . 15
- 6.3 AC Automata . . . 15
- 6.4 Suffix Automata . . . 15
- 6.5 Z-algorithm . . . 16
- 6.6 Knuth-Morris-Pratt Algorithm 16
- 6.7 Manacher's Algorithm . . . 16
- 6.8 Minimum Rotation . . . 17
- 6.9 Rolling Hash . . . 17
- 6.10 Another Hash . . . 17
- 6.11 字典樹 . . . 17
- 6.12 回文樹 . . . 17

7 Geometry 17
- 7.1 Point . . . 17
- 7.2 Line . . . 18
- 7.3 Circle . . . 18
- 7.4 點線距 . . . 18
- 7.5 PointInPolygon . . . 18
- 7.6 Intersection Of Line . . . 18
- 7.7 Intersection Of Circle . . . 18
- 7.8 Intersection Of Circle and Line 18
- 7.9 Convex Hull . . . 18
- 7.10 旋轉卡尺 . . . 19
- 7.11 Convex Hull Trick . . . 19
- 7.12 Area Of Circle and Polygon 19
- 7.13 Area Of Circle and Triangle 19
- 7.14 Area Of Sector . . . 19
- 7.15 切線 . . . 19
- 7.16 公切線 . . . 20
- 7.17 半平面交 . . . 20
- 7.18 最小包覆圓 . . . 20
- 7.19 圓聯集面積 . . . 20
- 7.20 多邊形聯集面積 . . . 20
- 7.21 Pick's Theory . . . 21

8 DP 21
- 8.1 區間 DP . . . 21
- 8.2 位數 DP . . . 21
- 8.3 SOS DP . . . 21
- 8.4 凸包優化 DP . . . 21
- 8.5 分治優化 DP . . . 22

9 Sorting 22
- 9.1 Merge Sort . . . 22
- 9.2 Quick Sort . . . 22

10 Miscellaneous 22
- 10.1 Discretization . . . 22
- 10.2 Mo's Algorithm . . . 22
- 10.3 CDQ 分治 . . . 22
- 10.4 Decimal . . . 23
- 10.5 Fraction . . . 23
- 10.6 Blessing . . . 23
- 10.7 Graph Paper . . . 24

1 Basis

1.1 Default Code

```cpp
// g++args: -std=c++17 -Wall -Wshadow -fsanitize=undefined
#pragma GCC optimize("O3,unroll-loops")
#pragma target optimize("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.2 Some Useful Primes

```cpp
/* 13331, 75577, 999983, 1097774749, 1076767633,
 * 100102021, 999997771, 1001010013, 1000512343,
 * 987654361, 999991231, 999888733, 98789101, 987777733,
 * 999991921, 1010101333, 1010102101, 1000000000039,
 * 1000000000000037, 2305843009213693951,
 * 4611686018427387847, 9223372036854775783,
 * 18446744073709551557 */
```

1.3 偽隨機 & MT19937

```cpp
// chrono::steady_clock::now().time_since_epoch().count();
mt19937 gen(/*seed*/ hash<string>()("ShanC_orz"));
mt19937_64 mt(/*seed*/ 114514);
int randint(int a, int b) {  // [a, b]
  return uniform_int_distribution<int>(a, b)(gen);
}
// srand(time(NULL)), rand() % (b - a + 1) + a;
```

1.4 對拍 code

```cpp
# chmod +x check.sh 給予執行權限
# 執行 ./check.sh
#!/bin/bash
g++ gen.cpp -o gen -O2    # 生測資
g++ sol.cpp -o sol -O2    # 你的扣
g++ slow.cpp -o slow -O2  # 很慢但會 AC 的扣

for ((i=1; ; ++i)); do
    ./gen > in
    ./sol < in > out
    ./slow < in > ans

    # -w 忽略空白字元 避免 Presentation Error
    if diff -w out ans > /dev/null; then
        echo "Pass: $i"
    else
        echo "WA on test $i"
        break
    fi
done
```

1.5 快讀快寫

```cpp
inline int read() {
  char c = getchar();
  int x = 0, f = 1;
  while (c < '0' || c > '9') {
    if (c == '-') f = -1;
    c = getchar();
  }
  while (c >= '0' && c <= '9')
    x = x * 10 + c - '0', c = getchar();
  return x * f;
}
inline void write(int x) {
  if (x < 0) putchar('-'), x = -x;
  if (x > 9) write(x / 10);
  putchar(x % 10 + '0');
}
```

1.6 怪怪的東西

```cpp
#define SECs ((double)clock() / CLOCKS_PER_SEC)
struct KeyHasher {
  size_t operator()(const Key& k) const {
    return k.first + k.second * 100000;
  }
};
typedef unordered_map<Key, int, KeyHasher> map_t;
__builtin_popcountll;  // 二進位有幾個1
__builtin_clzll;       // 左起第一個1之前0的個數
__builtin_parityll;    // popcount的奇偶性
__builtin_mul_overflow(a, b, &h)  // a*b是否溢位
__lg(x);               // floor(log2(x))
```

2 Data Structures

2.1 Disjoint Set (併查集)

```cpp
struct DSU {
  int n;
  vector<int> f, sz;
  DSU(int _n) : n(_n) {  // 初始化
    f.resize(n);
    sz.resize(n);
    for (int i = 0; i < n; i++) f[i] = i, sz[i] = 1;
  }
  int find(int x) {  // 返回 x 的根節點
    if (x == f[x]) return x;
    return f[x] = find(f[x]);
  }
  void merge(int x, int y) {  // 將 x 和 y 合併
    x = find(x), y = find(y);
    if (x == y) return;
    if (sz[x] < sz[y]) swap(x, y);  // 將小的併入大的
    sz[x] += sz[y];
    f[y] = x;
```

```
  }
};
```

2.2 Binary Indexed Tree

```
struct BIT {
  int n;
  vector<ll> bit;
  int lowbit(int x) { return x & -x; }
  BIT(int _n) : n(_n + 1) { bit = vector<ll>(_n + 1, 0); }
  void update(int x, ll v) {  // 將 x 的值加上 v
    for (; x < n; x += lowbit(x)) bit[x] += v;
  }
  ll query(int x) {  // 查詢區間 [1, x] 的總和
    ll ret = 0;
    for (; x; x -= lowbit(x)) ret += bit[x];
    return ret;
  }
  ll query(int l, int r) {  // 查詢區間 [l, r] 的總和
    return query(r) - query(l - 1);
  }
  int kth(int k) {
    int x = 0;
    for (int i = 1 << __lg(n); i; i >>= 1)
      if (x + i <= n && k >= bit[x + i - 1])
        x += i, k -= bit[x - 1];
    return x;
  }
};
```

2.3 二維 BIT

```
struct BIT {
  int n;
  vector<vector<ll>> bit;
  int lowbit(int x) { return x & -x; }
  BIT(int _n) : n(_n + 1) {
    bit.assign(n, vector<ll>(n, 0));
  }
  void update(int x, int y, ll val) {  // 更新 (x, y)
    for (int i = x; i < n; i += lowbit(i))
      for (int j = y; j < n; j += lowbit(j))
        bit[i][j] += val;
  }
  ll query(int x, int y) {  // 查詢 (1, 1) ~ (x, y) 的和
    ll ans = 0;
    for (int i = x; i; i -= lowbit(i))
      for (int j = y; j; j -= lowbit(j)) ans += bit[i][j];
    return ans;
  }
  ll range_query(int a, int b, int x, int y) {
    return query(x, y) - query(x, b - 1) -
           query(a - 1, y) + query(a - 1, b - 1);
  }
};
```

2.4 Segment Tree

```
#define cl (i << 1)
#define cr (i << 1 | 1)
struct SegmentTree {  // 1-based
  int n;
  vector<int> seg, tag;

  SegmentTree(int _n) : n(_n) {
    seg.assign(4 * n + 5, 0);
    tag.assign(4 * n + 5, 0);
  }
  void apply(int i, int v) {
    seg[i] += v;
    tag[i] += v;
  }
  void push(int i) {
    if (tag[i] != 0) {
      apply(cl, tag[i]);
      apply(cr, tag[i]);
      tag[i] = 0;
    }
  }
  void pull(int i) {
    seg[i] = max(seg[cl], seg[cr]);
  }
  void build(int i, int l, int r, const vector<int>& arr)
      {
    if (l == r) {
      seg[i] = arr[l];
      return;
    }
    int mid = (l + r) >> 1;
    build(cl, l, mid, arr);
    build(cr, mid + 1, r, arr);
    pull(i);
  }
  void update(int i, int l, int r, int ql, int qr, int v)
      {
    if (ql <= l && r <= qr) {
      apply(i, v);
      return;
    }
    push(i);
    int mid = (l + r) >> 1;
    if (ql <= mid) update(cl, l, mid, ql, qr, v);
    if (qr > mid) update(cr, mid + 1, r, ql, qr, v);
    pull(i);
  }
  int query(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return seg[i];
    push(i);
    int mid = (l + r) >> 1, ans = -1e9;
    if (ql <= mid) ans = max(ans, query(cl, l, mid, ql, qr
        ));
    if (qr > mid) ans = max(ans, query(cr, mid + 1, r, ql,
         qr));
    return ans;
  }
  void update(int ql, int qr, int v) { update(1, 1, n, ql,
       qr, v); }
  int query(int ql, int qr) { return query(1, 1, n, ql, qr
      ); }
};
```

2.5 Treap

```
struct Treap {
  int sz, val, pri, tag;
  Treap *l, *r;
  Treap(int _val) : sz(1), val(_val), tag(0) {
    l = r = NULL, pri = rand();  // 隨機產生優先度
  }
};
int size(Treap* a) { return a ? a->sz : 0; }
void pull(Treap* a) {
  a->sz = size(a->l) + size(a->r) + 1;
}
Treap* merge(Treap* a, Treap* b) {
  // val of a is always bigger than val of b
  if (!a || !b) return a ? a : b;
  if (a->pri > b->pri) {
    a->r = merge(a->r, b);
    pull(a);
    return a;
  } else {
    b->l = merge(a, b->l);
    pull(b);
    return b;
  }
}
void split(Treap* t, int k, Treap*& a, Treap*& b) {
  // a<k, b>=k
  if (!t) {
    a = b = NULL;
    return;
  }
  if (k <= t->val) {
    b = t;
    split(t->l, k, a, b->l);
    pull(b);
  } else {
    a = t;
    split(t->r, k, a->r, b);
    pull(a);
  }
}
Treap* add(Treap* t, int v) {
  Treap *val = new Treap(v), *l = NULL, *r = NULL;
  split(t, v, l, r);
  return merge(merge(l, val), r);
}
```

```
Treap* del(Treap* t, int v) {
  Treap *l, *mid, *r, *temp;
  split(t, v, l, temp);
  split(temp, v + 1, mid, r);
  return merge(l, r);
}
int position(Treap* t, int p) {  // base 1
  if (size(t->l) + 1 == p) return t->val;
  if (size(t->l) < p)
    return position(t->r, p - size(t->l) - 1);
  return position(t->l, p);
}
int query(Treap* t, int k) {  // num of >= k
  if (!t) return 0;
  if (t->val == k) return size(t->l) + 1;
  if (t->val > k) return query(t->l, k);
  return size(t->l) + 1 + query(t->r, k);
}
```

2.6 平板電腦

```
#include <bits/extc++.h>

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/rope>
#include <functional>
#include <queue>
using namespace std;
using namespace __gnu_cxx;
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update>
    set_t;
typedef cc_hash_table<int, int> umap_t;
typedef priority_queue<int> heap;

set_t s;  // Insert some entries into s.
s.insert(12), s.insert(505);
// The order of the keys should be: 12, 505.
assert(*s.find_by_order(0) == 12);
assert(*s.find_by_order(3) == 505);
// The order of the keys should be: 12, 505.
assert(s.order_of_key(12) == 0);
assert(s.order_of_key(505) == 1);
s.erase(12);  // Erase an entry.
// The order of the keys should be: 505.
assert(*s.find_by_order(0) == 505);
// The order of the keys should be: 505.
assert(s.order_of_key(505) == 0);
heap h1, h2;
h1.join(h2);
rope<char> r[2];
r[1] = r[0];  // persistenet
string t = "abc";
r[1].insert(0, t.c_str());
r[1].erase(1, 1);
cout << r[1].substr(0, 2);
```

3 Mathematics

3.1 公式們

$$\sum_{k=1}^{n} n = \frac{n(n+1)}{2}, \quad \sum_{k=1}^{n} n^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{k=1}^{n} n^3 = \left[\frac{n(n+1)}{2}\right]^2$$

$$\sum_{k=1}^{n} n^4 = n(1+n)(1+2n)(-1+3n+3n^2)/30$$

$$\sum_{k=1}^{n} n^5 = n^2(n+1)^2(2n^2+2n-1)/12$$

$$C_n = \frac{C_n^{2n}}{n+1} = \frac{(2n)!}{n!*(n+1)!}$$

錯位排列 $$DP_i = \begin{cases} 0 \vee 1, & i = 1 \vee 2 \\ (i-1)(DP_{i-1} \times DP_{i-2}), & i \geq 3 \\ = iDP_{i-1} + (-1)^i & \text{另一種關係式} \end{cases}$$

生成樹數量 n^{n-2}

$$P_k^n = \frac{n!}{k!(n-k)!}, \quad C_k^n = \frac{n!}{(n-k)!}, \quad H_k^n = C_{n-1}^{n+k-1}$$

$$\pi = \arccos(-1), \quad e^{i\theta} = \cos\theta + i\sin\theta, \quad pV = nTR$$

$$\varphi(n) = n\prod_{j=1}^{r}\left(1-\frac{1}{p_j}\right), n = p_1^{k_1}p_2^{k_2}\cdots p_r^{k_r}$$

$$a^{\varphi(m)} \equiv 1 (\bmod m), \quad a^b \equiv a^{b \bmod \varphi(m)-1} (\bmod m)$$

$$\varphi(m) = m-1, \text{When } m \text{ is Prime}$$

$$\mu(n) = \begin{cases} 1 & , n = 1 \\ (-1)^k & , n = p_1p_2\cdots p_k \text{ (沒平方項)} \\ 0 & , \text{otherwise} \end{cases}, \quad (f*g)(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right)$$

$$[\gcd(i,j) = 1] = \sum_{d|\gcd(i,j)} \mu(d), \quad \sigma(n) = \sum_{d|n} d$$

$$\epsilon(n) = [n = 1], \quad \mathrm{Id}(n) = n, \quad 1(n) = 1$$

$$\mu * 1 = \epsilon, \quad \mu * f = g, \quad \varphi = \mu * \mathrm{Id}, \quad \varphi * 1 = \mathrm{Id}$$

$$\sum_{i=1}^{N}\sum_{j=1}^{M}[\gcd(i,j) = 1] = \sum_{d=1}^{N}\mu(d)\left\lfloor\frac{M}{d}\right\rfloor\left\lfloor\frac{N}{d}\right\rfloor$$

$$\sum_{i=1}^{N}\sum_{j=1}^{M}\gcd(i,j) = \sum_{T=1}^{\min(N,M)}\phi(T)\left\lfloor\frac{N}{T}\right\rfloor\left\lfloor\frac{M}{T}\right\rfloor$$

3.2 階乘 & 模逆元

```
ll fac[MXN], inv[MXN];
void init() {
  fac[0] = 1;  // 0! = 1
  for (ll i = 1; i < MXN; i++)
    fac[i] = fac[i - 1] * i % MOD;  // n! = n * (n-1)!
  inv[MXN - 1] = power(fac[MXN - 1], MOD - 2);
  for (ll i = MXN - 2; i >= 0; i--)
    inv[i] = inv[i + 1] * (i + 1) % MOD;  // (n!)^(-1)
}
```

3.3 GCD & LCM

```
ll gcd(ll a, ll b) {
  if (b && a)  // 輾轉先除法 -> 交叉取餘數直到不能再取
    while ((a %= b) && (b %= a)) {}
  return a + b;
}
ll lcm(ll a, ll b) { return a * b / gcd(a, b); }
```

3.4 EXGCD

$ax + by = gcd(a, b)$

```
int exgcd(int a, int b, ll& x, ll& y) {
  if (b == 0) {
    x = 1, y = 0;
    return a;
  }
  int now = exgcd(b, a % b, y, x);
  y -= a / b * x;
  return now;
}
```

3.5 Fast Power

```
ll power(ll x, ll y, ll mod) {
  ll ret = 1;
  while (y) {
    if (y & 1) ret = ret * x % mod;
    x = x * x % mod, y >>= 1;
  }
  return ret;
}
```

3.6 矩陣乘法

```
struct Mat {  // Mat t(r, c);
  ll a[200][200], r, c;
  Mat(int _r, int _c) : r(_r), c(_c) {
    memset(a, 0, sizeof(a));
  }
  void build() {  // 單位矩陣
    for (int i = 0; i < r; ++i) a[i][i] = 1;
  }
};
Mat operator*(Mat x, Mat y) {
  Mat z(x.r, y.c);
  for (int i = 0; i < x.r; ++i)
    for (int j = 0; j < x.c; ++j)
      for (int k = 0; k < y.c; ++k)
        (z.a[i][j] += x.a[i][k] * y.a[k][j] % MOD) %= MOD;
  return z;
}
```

3.7 多項式 and 操作

```
using poly = vector<i64>;
poly operator*(poly A, poly B) {
  return convolution<mod, G>(A, B);
}
poly operator-(i64 r, poly A) {
  r %= mod;
  r = (r + mod) % mod;
  for (auto& c : A) {
    c %= mod;
    c = (mod - c) % mod;
  }
  A[0] = (A[0] + r) % mod;
  return A;
}
poly operator+(i64 r, poly A) {
  r %= mod;
  r = (r + mod) % mod;
  A[0] = (A[0] + r) % mod;
  return A;
}
poly operator*(i64 r, poly A) {
  for (auto& c : A) {
    c = r * c % mod;
    c = (c + mod) % mod;
  }
  return A;
}
poly operator-(poly A, poly B) {
  int n = max(A.size(), B.size());
  A.resize(n), B.resize(n);
  for (int i = 0; i < n; ++i) {
    A[i] = (A[i] - B[i]) % mod;
    A[i] = (A[i] + mod) % mod;
  }
  return A;
}
poly operator+(poly A, poly B) {
  int n = max(A.size(), B.size());
  A.resize(n), B.resize(n);
  for (int i = 0; i < n; ++i) {
    A[i] = (A[i] + B[i]) % mod;
    A[i] = (A[i] + mod) % mod;
  }
  return A;
}
poly operator%(poly A, int n) {
  A.resize(n);
  return A;
}

poly derive(poly P) {
  if (!P.size()) return P;
  for (int i = 0; i < P.size(); ++i) {
    P[i] = i * P[i] % mod;
    P[i] = (P[i] + mod) % mod;
  }
  P.erase(P.begin());
  return P;
}
poly integr(poly P) {
  if (!P.size()) return P;
  for (int i = 0; i < P.size(); ++i) {
    P[i] = P[i] * inv(i + 1) % mod;
    P[i] = (P[i] + mod) % mod;
  }
  P.insert(P.begin(), 0);
  return P;
}
// nlogn f(x) -> f(x + k)
auto shift = [&](vector<i64> f, i64 k) {
  k %= mod;
  k += mod;
  k %= mod;
  int n = f.size() - 1;
  vector<i64> g(n + 1);
  for (int i = 0; i <= n; ++i) {
    f[i] = f[i] * fac[i] % mod;
    g[n - i] = fpow(k, i) * inv(fac[i]) % mod;
    // x^(-n) -> x^(0)
  }
  auto h = convolution<mod, 3>(f, g);
  h.erase(h.begin(), h.begin() + n);
  for (int i = 0; i <= n; ++i)
    h[i] = h[i] * inv(fac[i]) % mod;
  return h;
};
poly log(poly P) {
  return integr(derive(P) * inverse(P));
}
poly inverse(poly A) {
  int n = A.size();
  int tn = 1;
  poly Q(tn);
  Q[0] = inv(A[0]);
  while (tn < n) {
    tn *= 2;
    Q = Q * (2 - A % tn * Q % tn);
    Q = Q % tn;
  }
  return Q;
}
poly exp(poly P) {
  int n = P.size();
  int tn = 1;
  poly Q(tn);
  Q[0] = 1;
  while (tn < n) {
    tn *= 2;
    Q = Q * (1 + P % tn - log(Q) % tn);
    Q = Q % tn;
  }
  return Q;
}
```

3.8 FFT

使用前要先跑過 pre_fft()；n 必須是 2^k。

```
const int MAXN = 262144 << 1;  // (must be 2^k)
// before any usage, run pre_fft() first
// typedef long double ld;
typedef complex<ld> cplx;  // real(), imag()
const ld PI = acosl(-1);
const cplx I(0, 1);
cplx omega[MAXN + 1];
void pre_fft() {
  for (int i = 0; i <= MAXN; i++)
    omega[i] = exp(i * 2 * PI / MAXN * I);
}
void fft(int n, cplx a[], bool inv = false) {
  int basic = MAXN / n;
  int theta = basic;
  for (int m = n; m >= 2; m >>= 1) {
    int mh = m >> 1;
    for (int i = 0; i < mh; i++) {
      cplx w = omega[inv ? MAXN - (i * theta % MAXN)
                         : i * theta % MAXN];
      for (int j = i; j < n; j += m) {
        int k = j + mh;
        cplx x = a[j] - a[k];
        a[j] += a[k];
        a[k] = w * x;
      }
    }
    theta = (theta * 2) % MAXN;
  }
  int i = 0;
  for (int j = 1; j < n - 1; j++) {
    for (int k = n >> 1; k > (i ^= k); k >>= 1) {}
    if (j < i) swap(a[i], a[j]);
  }
  if (inv)
    for (i = 0; i < n; i++) a[i] /= n;
}
cplx arr[MAXN + 1];
inline void mul(int _n, ll a[], int _m, ll b[],
                ll ans[]) {
  int n = 1, sum = _n + _m - 1;
  while (n < sum) n <<= 1;
  for (int i = 0; i < n; i++) {
    double x = (i < _n ? a[i] : 0),
           y = (i < _m ? b[i] : 0);
    arr[i] = complex<double>(x + y, x - y);
  }
  fft(n, arr);
  for (int i = 0; i < n; i++) arr[i] = arr[i] * arr[i];
  fft(n, arr, true);
```

```
  for (int i = 0; i < sum; i++)
    ans[i] = (long long int)(arr[i].real() / 4 + 0.5);
}
```

3.9 NTT

```
// Remember coefficient are mod P
/* p=a*2^n+1
   n    2^n         p          a    root
   16   65536       65537      1    3
   20   1048576     7340033    7    3 */
// (must be 2^k)
template <LL P, LL root, int MAXN>
struct NTT {
  static LL bigmod(LL a, LL b) {
    LL res = 1;
    for (LL bs = a; b; b >>= 1, bs = (bs * bs) % P)
      if (b & 1) res = (res * bs) % P;
    return res;
  }
  static LL inv(LL a, LL b) {
    if (a == 1) return 1;
    return (((LL)(a - inv(b % a, a)) * b + 1) / a) % b;
  }
  LL omega[MAXN + 1];
  NTT() {
    omega[0] = 1;
    LL r = bigmod(root, (P - 1) / MAXN);
    for (int i = 1; i <= MAXN; i++)
      omega[i] = (omega[i - 1] * r) % P;
  }
  // n must be 2^k
  void tran(int n, LL a[], bool inv_ntt = false) {
    int basic = MAXN / n, theta = basic;
    for (int m = n; m >= 2; m >>= 1) {
      int mh = m >> 1;
      for (int i = 0; i < mh; i++) {
        LL w = omega[i * theta % MAXN];
        for (int j = i; j < n; j += m) {
          int k = j + mh;
          LL x = a[j] - a[k];
          if (x < 0) x += P;
          a[j] += a[k];
          if (a[j] > P) a[j] -= P;
          a[k] = (w * x) % P;
        }
      }
      theta = (theta * 2) % MAXN;
    }
    int i = 0;
    for (int j = 1; j < n - 1; j++) {
      for (int k = n >> 1; k > (i ^= k); k >>= 1);
      if (j < i) swap(a[i], a[j]);
    }
    if (inv_ntt) {
      LL ni = inv(n, P);
      reverse(a + 1, a + n);
      for (i = 0; i < n; i++) a[i] = (a[i] * ni) % P;
    }
  }
};
const LL P = 2013265921, root = 31;
const int MAXN = 4194304;
NTT<P, root, MAXN> ntt;
```

3.10 質數表

```
// notPrime, 最大質因數, 質因數數量
int notPrime[MXN], fac[MXN], num[MXN];
void getPrimes() {
  notPrime[1] = 1, fac[1] = num[1];
  for (int i = 2; i < MXN; ++i) {
    if (notPrime[i]) continue;
    for (int j = i; j < MXN; j += i) {
      if (i != j) notPrime[j] = 1;
      fac[j] = i, num[j]++;
    }
  }
}
vector<int> ret;  // 質因數分解
void div(int x) {
  for (; x > 1; x /= fac[x]) ret.push_back(fac[x]);
}
```

3.11 歐拉函數

$\varphi(x) =$ 所有小於等於 x 且和 x 互質的數的數量

```
int _phi(int n) {  // O(sqrtN)
  int res = n, a = n;
  for (int i = 2; i * i <= a; i++) {
    if (a % i == 0) {
      res = res / i * (i - 1);
      while (a % i == 0) a /= i;
    }
  }
  if (a > 1) res = res / a * (a - 1);
  return res;
}
```

```
int phi[MXN]; // 線性篩打表
void phi_table(int n) {
  is_prime[0] = is_prime[1] = false;
  phi[1] = 1;
  for(int i = 2; i <= n; ++i) {
    if(is_prime[i]) {
      primes.push_back(i);
      phi[i] = i - 1;
    }
    for(int p : primes){
      if(1LL*i*p > n) break;
      is_prime[i * p] = false;
      if(i % p == 0) {
        phi[i * p] = phi[i] * p;
        break;
      } else {
        phi[i * p] = phi[i] * (p - 1);
      }
    }
  }
}
```

3.12 Miller-Rabin Algorithm

$$Magic = \begin{cases} \{2,7,61\}, & n < 4,759,123,141 \\ \{2,13,23,1662803\}, & n < 1,122,004,669,633 \\ \{2,3,5,7,11,13\}, & n < 3,474,749,660,383 \\ \{2,325,9375,28178, & n < 2^{64} \\ 450775,9780504,1795265022\} & \end{cases}$$

```
ll mul(ll x, ll y, ll mod) {
  ll ret = x * y - (ll)((long double)x / mod * y) * mod;
  return ret < 0 ? ret + mod : ret;
  // return x * y % mod; // for __int128
}
ll magic[] = {2, 7, 61};  // 這邊填入要用於測試的數列
ll S = 3;                 // 測試的數列數字的數量
bool witness(ll a, ll n, ll u, int t) {
  if (!a) return 0;
  ll x = power(a, u, n);
  for (int i = 0; i < t; i++) {
    ll nx = mul(x, x, n);
    if (nx == 1 && x != 1 && x != n - 1) return 1;
    x = nx;
  }
  return x != 1;
}
bool miller_rabin(ll n) {
  int s = S;  // magic number size
  // iterate s times of witness on n
  if (n < 2) return false;
  if (!(n & 1)) return (n == 2);

  // 將 n - 1 寫成 u * (2 ^ t) 的形式
  ll u = n - 1;
  int t = 0;
  while (!(u & 1)) u >>= 1, t++;

  while (s--) {
    ll a = magic[s] % n;
    if (witness(a, n, u, t)) return false;
  }
  return true;
}
```

3.13 Pollard's Rho Algorithm

```
vector<ll> ret;
ll f(ll x, ll c, ll mod) {
  return (mul(x, x, mod) + c) % mod;
}
ll pollard_rho(ll n) {
  ll c = 1, x = 0, y = 0, p = 2, q, t = 0;
  while (t++ % 128 || gcd(p, n) == 1) {
    if (x == y) c++, y = f(x = 2, c, n);
    if (q = mul(p, abs(x - y), n)) p = q;
    x = f(x, c, n);
    y = f(f(y, c, n), c, n);
  }
  return gcd(p, n);
}
void fact(ll x) {  // 透過遞迴找質數
  if (miller_rabin(x)) {
    ret.push_back(x);
    return;
  }
  ll f = pollard_rho(x);
  fact(f);
  fact(x / f);
}
```

3.14 高斯消去法

```
/**
 * mat[n][m] 為 n 個 m 元一次連例方程式的增廣矩陣
 * ans[m] 輸出 x_m 的解
 * pivot[n] 紀錄第 n 列的領導係數的位置
 * 若無解則回傳 false
 */
ll mat[MXN][MXN], ans[MXN];
int pivot[MXN];

/**
 * 回傳 true 代表有解，false 代表無解
 */
bool gaussian(int n, int m) {
  fill(pivot, pivot + m, -1);
  int row = 0;
  for (int col = 0; col < m && row < n; col++) {
    int sel = -1;
    for (int i = row; i < n;
         i++) {  // 找這一欄係數不為 0 的列
      if (mat[i][col] != 0) {
        sel = i;
        break;
      }
    }
    if (sel ==
        -1)  // 全是自由變數，直接跳過處理下一個變數
      continue;
    // 把找到的那一列換到當前 row 的位置
    for (int j = col; j <= m; j++)
      swap(mat[sel][j], mat[row][j]);

    // 歸一化：把主元 (mat[row][col]) 變成 1
    for (int j = col; j <= m; j++)
      mat[row][j] = mat[row][j] / mat[row][col];

    for (int i = 0; i < n; i++) {  // 消去
      if (i != row && mat[i][col] != 0) {
        ll factor = mat[i][col];
        for (int j = col; j <= m; j++)
          mat[i][j] = mat[i][j] - factor * mat[row][j];
      }
    }
    // 紀錄這個變數是由第 row 列解出來的
    // 處理下一列
    pivot[col] = row++;
  }
  // 檢查無解的情況
  // 剩下的列 (從 row 到 n-1)，左邊係數應該全被消成 0 了
  // 如果常數項 (mat[i][m]) 不為 0，表示 0 = k
  // (k!=0)，即無解
  for (int i = row; i < n; i++) {
    if (mat[i][m] != 0) return false;
  }
  for (int j = 0; j < m; j++) {  // 填入答案
    if (pivot[j] != -1)  // 對應的主元列的最右邊常數即為解
      ans[j] = mat[pivot[j]][m];
    else ans[j] = 0;
  }
  return true;
}
```

3.15 約瑟夫問題

```
int josephus(int n, int m) {  // n 人每 m 次
  int ans = 0;
  for (int i = 1; i <= n; ++i) ans = (ans + m) % i;
  return ans;
}
```

3.16 莫比烏斯函數

```
vector<int> pri;
bool not_prime[MXN];
int mu[MXN];

void pre(int n) {
  mu[1] = 1;
  for (int i = 2; i <= n; ++i) {
    if (!not_prime[i]) {
      mu[i] = -1;
      pri.push_back(i);
    }
    for (int pri_j : pri) {
      if (i * pri_j > n) break;
      not_prime[i * pri_j] = true;
      if (i % pri_j == 0) {
        mu[i * pri_j] = 0;
        break;
      }
      mu[i * pri_j] = -mu[i];
    }
  }
}
```

3.17 離散對數

```
// gcd(a, n) = 1, 算 a^x == b (mod n)
ll BSGS(ll a, ll b, ll n) {
  a %= n, b %= n;
  if (b == 1) return 0;    // a^0 = 1
  ll m = (ll)sqrt(n) + 1;  // 保險一點
  unordered_map<ll, ll> mp;

  // 拆成 a^{i * m} == b * a^j (mod n)
  ll baby = b;
  for (ll j = 0; j <= m; j++) {
    mp[baby] = j;
    baby = baby * a % n;
  }

  ll step = fpow(a, m, n);
  ll giant = step;
  for (ll i = 1; i <= m; i++) {  // 計算 (a^m)^i
    if (mp.count(giant)) {
      ll j = mp[giant], res = i * m - j;
      if (res >= 0) return res;
    }
    giant = giant * step % n;
  }

  return -1;
}
```

3.18 離散根號

```
void calcH(ll& t, ll& h, const ll p) {
  ll tmp = p - 1;
  for (t = 0; (tmp & 1) == 0; tmp /= 2) t++;
  h = tmp;
}
// solve equation x^2 mod p = a
bool solve(ll a, ll p, ll& x, ll& y) {
  if (p == 2) {
    x = y = 1;
    return true;
  }
  int p2 = p / 2, tmp = power(a, p2, p);
  if (tmp == p - 1) return false;
```

```
  if ((p + 1) % 4 == 0) {
    x = power(a, (p + 1) / 4, p);
    y = p - x;
    return true;
  } else {
    ll t, h, b, pb;
    calcH(t, h, p);
    if (t >= 2) {
      do {
        b = rand() % (p - 2) + 2;
      } while (power(b, p / 2, p) != p - 1);
      pb = power(b, h, p);
    }
    int s = power(a, h / 2, p);
    for (int step = 2; step <= t; step++) {
      int ss = (((ll)(s * s) % p) * a) % p;
      for (int i = 0; i < t - step; i++)
        ss = mul(ss, ss, p);
      if (ss + 1 == p) s = (s * pb) % p;
      pb = ((ll)pb * pb) % p;
    }
    x = ((ll)s * a) % p;
    y = p - x;
  }
  return true;
}
```

3.19 XOR Basis

```
const int LOGN = 31;
vector<int> find_basis(vector<int>& arr) {
  vector<int> basis(LOGN);
  for (int& x : arr) {
    for (int i = LOGN - 1; i >= 0; i--) {
      if ((x & (1 << i)) == 0) continue;
      if (!basis[i]) {
        basis[i] = x;
        break;
      } else x ^= basis[i];
    }
  }
  return basis;
}
bool check(int x) {
  for (int i = LOGN; i >= 0; i--) {
    if (x & (1 << i)) {
      if (!basis[i]) return false;
      x ^= basis[i];
    }
  }
  return true;
}
```

4 Graph

4.1 建圖方法

2-SAT 建圖
$$\begin{cases} \neg a \vee b \text{ 轉} a \to b \text{ 和} \neg a \to \neg b \\ a \vee b \text{ 轉} \neg a \to b \text{ 和} \neg b \to a \\ \neg a \vee \neg b \text{ 轉} a \to \neg b \text{ 和} a \to \neg b \end{cases}$$

2-SAT 求解
$$\begin{cases} \text{scc.bln[i] > scc.bln[not i] : } x_i \text{ 為 True} \\ \text{scc.bln[not i] > scc.bln[i] : } x_i \text{ 為 False} \\ \text{否則無解} \end{cases}$$

差分約束建圖
$$\begin{cases} x_a - x_b \le c & \text{add(b, a, c)} \\ x_a - x_b \ge c & \text{add(a, b, -c)} \\ x_a = x_b & \text{add(a, b, 0) and add(b, a, 0)} \end{cases}$$

4.2 Topological Sort

```
vector<int> edge[MAXN], ans;
int deg[MAXN];      // in degree
void topo(int n) {  // 1-base
  queue<int> que;
  for (int i = 1; i <= n; i++)
    if (!deg[i]) que.push(i);
  while (!que.empty()) {
    int u = que.front();
    que.pop();
    ans.push_back(u);
    // 結果存 ans
    for (int v : edge[u]) {
      deg[v]--;
      if (!deg[v]) que.push(v);
    }
  }
}
```

4.3 Tarjan's Algorithm for BCC

```
struct BccVertex {
  int n, nScc, step, dfn[MXN], low[MXN];
  vector<int> E[MXN], sccv[MXN];
  int top, stk[MXN];
  void init(int _n) {
    n = _n;
    nScc = step = 0;
    for (int i = 0; i < n; i++)
      E[i].clear();
  }
  void addEdge(int u, int v) {
    E[u].PB(v);
    E[v].PB(u);
  }
  void DFS(int u, int f) {
    dfn[u] = low[u] = step++;
    stk[top++] = u;
    for (auto v : E[u]) {
      if (v == f) continue;
      if (dfn[v] == -1) {
        DFS(v, u);
        low[u] = min(low[u], low[v]);
        if (low[v] >= dfn[u]) {
          int z;
          sccv[nScc].clear();
          do {
            z = stk[--top];
            sccv[nScc].PB(z);
          } while (z != v);
          sccv[nScc++].PB(u);
        }
      } else low[u] = min(low[u], dfn[v]);
    }
  }
  vector<vector<int>> solve() {
    vector<vector<int>> res;
    for (int i = 0; i < n; i++)
      dfn[i] = low[i] = -1;
    for (int i = 0; i < n; i++)
      if (dfn[i] == -1) {
        top = 0;
        DFS(i, i);
      }
    REP(i, nScc) res.PB(sccv[i]);
    return res;
  }
} graph;
```

4.4 Bellmen-Ford Algorithm

```
struct Edge {
  int from, to, weight;
} edge[MAXN];
int dis[MAXN];
void bellmen_ford(int n, int m) {
  dis[1] = 0;  // 起點的距離一定是 0
  for (int j = 0; j < n - 1;
       j++) {  // n 個節點要跑 n - 1 次
    for (int i = 0; i < m; i++) {  // 對於所有邊都嘗試鬆弛
      if (dis[edge[i].to] >
          dis[edge[i].from] + edge[i].weight)
        dis[edge[i].to] =
            dis[edge[i].from] + edge[i].weight;
    }
  }
}
```

4.5 Dijkstra's Algorithm

```
vector<pii> vec[N];  // vec[u] = {v, w}: u 為起點 v 為終點
                     // w 為路權
bool vis[N];                    // 紀錄該節點是否走過
vector<int> dijkstra(int s) {  // 起點
```

```
  vector<int> dis(N);
  for (int i = 0; i < N; i++)  // 初始化
    dis[i] =
        INF;  // 值要設為比可能的最短路徑權重還要大的值
  dis[s] = 0;
  priority_queue<pii, vector<pii>, greater<pii>>
      pq;  // 以小到大排序
  // {w, v}: w 為路權 v 為節點
  pq.push({dis[s], s});
  while (pq.empty() == 0) {
    int u = pq.top().second;
    pq.pop();
    if (vis[u])  // 走過的不要再走
      continue;
    vis[u] = 1;
    for (auto i : vec[u]) {
      int v = i.first, w = i.second;
      if (dis[u] + w < dis[v]) {  // 鬆弛
        dis[v] = dis[u] + w;
        pq.push({dis[v], v});
      }
    }
  }
  return dis;
}
```

4.6 Dinic's Algorithm for Maximum Flow

```
#define SZ(x) ((int)x.size())
#define PB push_back
struct Dinic {
  struct Edge {
    int v, f, re;
  };
  int n, s, t, level[MXN];
  vector<Edge> E[MXN];
  void init(int _n, int _s, int _t) {
    n = _n;
    s = _s;
    t = _t;
    for (int i = 0; i < n; i++) E[i].clear();
  }
  void add_edge(int u, int v, int f) {
    E[u].PB({v, f, SZ(E[v])});
    E[v].PB({u, 0, SZ(E[u]) - 1});
  }
  bool BFS() {
    for (int i = 0; i < n; i++) level[i] = -1;
    queue<int> que;
    que.push(s);
    level[s] = 0;
    while (!que.empty()) {
      int u = que.front();
      que.pop();
      for (auto it : E[u]) {
        if (it.f > 0 && level[it.v] == -1) {
          level[it.v] = level[u] + 1;
          que.push(it.v);
        }
      }
    }
    return level[t] != -1;
  }
  int DFS(int u, int nf) {
    if (u == t) return nf;
    int res = 0;
    for (auto &it : E[u]) {
      if (it.f > 0 && level[it.v] == level[u] + 1) {
        int tf = DFS(it.v, min(nf, it.f));
        res += tf;
        nf -= tf;
        it.f -= tf;
        E[it.v][it.re].f += tf;
        if (nf == 0) return res;
      }
    }
    if (!res) level[u] = -1;
    return res;
  }
  int flow(int res = 0) {
    while (BFS()) res += DFS(s, 2147483647);
    return res;
  }
} flow;
/*
    查找二分圖配對點:
    在 add_edge 時寫上 index[u].push_back({v,
   flow.E[u].size()});

    在 flow 完之後寫上這個
    for (int i = 1; i <= n; i++) { // 遍歷所有節點
        for (pair<int, int> j : index[i]) {
            if (flow.E[i][j.second - 1].f == 0)
                cout << i << ' ' << j.first << '\n';
        }
    }
 */
```

4.7 Dominator Tree

```
#define REP(i, s, e) for (int i = (s); i <= (e); i++)
#define REPD(i, s, e) for (int i = (s); i >= (e); i--)
struct DominatorTree {  // O(N)
  int n, s;
  vector<int> g[MAXN], pred[MAXN];
  vector<int> cov[MAXN];
  int dfn[MAXN], nfd[MAXN], ts;
  int par[MAXN];  // idom[u] s到u的最後一個必經點
  int sdom[MAXN], idom[MAXN];
  int mom[MAXN], mn[MAXN];
  inline bool cmp(int u, int v) {
    return dfn[u] < dfn[v];
  }
  int eval(int u) {
    if (mom[u] == u) return u;
    int res = eval(mom[u]);
    if (cmp(sdom[mn[mom[u]]], sdom[mn[u]]))
      mn[u] = mn[mom[u]];
    return mom[u] = res;
  }
  void init(int _n, int _s) {
    ts = 0;
    n = _n;
    s = _s;
    REP(i, 1, n) g[i].clear(), pred[i].clear();
  }
  void addEdge(int u, int v) {
    g[u].push_back(v);
    pred[v].push_back(u);
  }
  void dfs(int u) {
    ts++;
    dfn[u] = ts;
    nfd[ts] = u;
    for (int v : g[u])
      if (dfn[v] == 0) {
        par[v] = u;
        dfs(v);
      }
  }
  void build() {
    REP(i, 1, n) {
      dfn[i] = nfd[i] = 0;
      cov[i].clear();
      mom[i] = mn[i] = sdom[i] = i;
    }
    dfs(s);
    REPD(i, n, 2) {
      int u = nfd[i];
      if (u == 0) continue;
      for (int v : pred[u])
        if (dfn[v]) {
          eval(v);
          if (cmp(sdom[mn[v]], sdom[u]))
            sdom[u] = sdom[mn[v]];
        }
      cov[sdom[u]].push_back(u);
      mom[u] = par[u];
      for (int w : cov[par[u]]) {
        eval(w);
        if (cmp(sdom[mn[w]], par[u])) idom[w] = mn[w];
        else idom[w] = par[u];
      }
      cov[par[u]].clear();
    }
    REP(i, 2, n) {
```

```
      int u = nfd[i];
      if (u == 0) continue;
      if (idom[u] != sdom[u]) idom[u] = idom[idom[u]];
    }
  }
} domT;
```

4.8 Edmonds Blossom Algorithm

```
namespace blossom {
#define d(x) (lab[x.u] + lab[x.v] - 2 * e[x.u][x.v].w)
#define all(x) x.begin(), x.end()
const ll N = 403 * 2;
const ll inf = 1e18;
struct Q {
  ll u, v, w;
} e[N][N];
vector<ll> p[N];
ll n, m = 0, id, h, t, lk[N], sl[N], st[N], f[N], b[N][N],
      s[N], ed[N], q[N], lab[N];
void upd(ll u, ll v) {
  if (!sl[v] || d(e[u][v]) < d(e[sl[v]][v])) sl[v] = u;
}
void ss(ll v) {
  sl[v] = 0;
  for (ll u = 1; u <= n; ++u)
    if (e[u][v].w > 0 && st[u] != v && !s[st[u]])
      upd(u, v);
}
void ins(ll u) {
  if (u <= n) q[++t] = u;
  else
    for (ll v : p[u]) ins(v);
}
void ch(ll u, ll w) {
  st[u] = w;
  if (u > n)
    for (ll v : p[u]) ch(v, w);
}
ll gr(ll u, ll v) {
  if ((v = find(all(p[u]), v) - p[u].begin()) & 1) {
    reverse(1 + all(p[u]));
    return (ll)p[u].size() - v;
  }
  return v;
}
void stm(ll u, ll v) {
  lk[u] = e[u][v].v;
  if (u <= n) return;
  Q w = e[u][v];
  ll x = b[u][w.u], y = gr(u, x);
  for (ll i = 0; i < y; ++i) stm(p[u][i], p[u][i ^ 1]);
  stm(x, v);
  rotate(p[u].begin(), y + all(p[u]));
}
void aug(ll u, ll v) {
  ll w = st[lk[u]];
  stm(u, v);
  if (!w) return;
  stm(w, st[f[w]]);
  aug(st[f[w]], w);
}
ll lca(ll u, ll v) {
  for (id++; u | v; swap(u, v)) {
    if (!u) continue;
    if (ed[u] == id) return u;
    ed[u] = id;
    if (u = st[lk[u]]) u = st[f[u]];  // =, not ==
  }
  return 0;
}
void add(ll u, ll a, ll v) {
  ll x = n + 1;
  while (x <= m && st[x]) ++x;
  if (x > m) ++m;
  lab[x] = s[x] = st[x] = 0;
  lk[x] = lk[a];
  p[x].clear();
  p[x].push_back(a);
#define op(q)                                          \
  for (ll i = q, j = 0; i != a; i = st[f[j]])          \
  p[x].push_back(i), p[x].push_back(j = st[lk[i]]), \
      ins(j)  // also not ==
  op(u);
  reverse(1 + all(p[x]));
  op(v);
  ch(x, x);
  for (ll i = 1; i <= m; ++i) e[x][i].w = e[i][x].w = 0;
  fill(b[x], b[x] + n + 1, 0);
  for (ll u : p[x]) {
    for (ll v = 1; v <= m; ++v)
      if (!e[x][v].w || d(e[u][v]) < d(e[x][v]))
        e[x][v] = e[u][v], e[v][x] = e[v][u];
    for (ll v = 1; v <= n; ++v)
      if (b[u][v]) b[x][v] = u;
  }
  ss(x);
}
void ex(ll u) {
  for (ll x : p[u]) ch(x, x);
  ll a = b[u][e[u][f[u]].u], r = gr(u, a);
  for (ll i = 0; i < r; i += 2) {
    ll x = p[u][i], y = p[u][i + 1];
    f[x] = e[y][x].u;
    s[x] = 1;
    s[y] = 0;
    sl[x] = 0;
    ss(y);
    ins(y);
  }
  s[a] = 1;
  f[a] = f[u];
  for (ll i = r + 1; i < p[u].size(); ++i)
    s[p[u][i]] = -1, ss(p[u][i]);
  st[u] = 0;
}
bool on(const Q &e) {
  ll u = st[e.u], v = st[e.v], a;
  if (s[v] == -1) {
    f[v] = e.u, s[v] = 1, a = st[lk[v]],
    sl[v] = sl[a] = s[a] = 0, ins(a);
  } else if (!s[v]) {
    a = lca(u, v);
    if (!a) return aug(u, v), aug(v, u), 1;
    else add(u, a, v);
  }
  return 0;
}
bool bfs() {
  fill(s, s + m + 1, -1);
  fill(sl, sl + m + 1, 0);  // s is filled with -1
  h = 1, t = 0;
  for (ll i = 1; i <= m; ++i)
    if (st[i] == i && !lk[i]) f[i] = s[i] = 0, ins(i);
  if (h > t) return 0;
  while (1) {
    while (h <= t) {
      ll u = q[h++];
      if (s[st[u]] != 1) {
        for (ll v = 1; v <= n; ++v)
          if (e[u][v].w > 0 && st[u] != st[v]) {
            if (d(e[u][v])) upd(u, st[v]);
            else if (on(e[u][v])) return 1;
          }
      }
    }
    ll x = inf;
    for (ll i = n + 1; i <= m; ++i)
      if (st[i] == i && s[i] == 1) x = min(x, lab[i] / 2);
    for (ll i = 1; i <= m; ++i)
      if (st[i] == i && sl[i] && s[i] != 1)
        x = min(x, d(e[sl[i]][i]) >> s[i] + 1);
    for (ll i = 1; i <= n; ++i)
      if (~s[st[i]])
        if ((lab[i] += (s[st[i]] * 2 - 1) * x) <= 0)
          return 0;
    for (ll i = n + 1; i <= m; ++i)
      if (st[i] == i && ~s[st[i]])
        lab[i] += (2 - 4 * s[st[i]]) * x;
    h = 1, t = 0;
    for (ll i = 1; i <= m; ++i)
      if (st[i] == i && sl[i] && st[sl[i]] != i &&
          !d(e[sl[i]][i]) && on(e[sl[i]][i]))
        return 1;
    for (ll i = n + 1; i <= m; ++i)
      if (st[i] == i && s[i] == 1 && !lab[i]) ex(i);
  }
}
```

```
pair<ll, vector<array<ll, 2>>> solve(
    ll N, vector<array<ll, 3>> edges) {
  for (auto &edge : edges) ++edge[0], ++edge[1];
  fill(ed, ed + m + 1, 0);
  fill(lk, lk + m + 1, 0);
  n = m = N;
  id = 0;
  iota(st + 1, st + n + 1, 1);
  ll wm = 0, weight = 0;
  for (ll i = 1; i <= n; ++i)
    for (ll j = 1; j <= n; ++j) e[i][j] = {i, j, 0};
  for (auto edge : edges) {
    int u = edge[0], v = edge[1];
    ll w = edge[2];
    wm = max(wm,
             e[v][u].w = e[u][v].w = max(e[u][v].w, w));
  }
  for (ll i = 1; i <= n; ++i) p[i].clear();
  for (ll i = 1; i <= n; ++i)
    for (ll j = 1; j <= n; ++j) b[i][j] = i == j ? i : 0;
  fill_n(lab + 1, n, wm);
  while (bfs()) {}
  vector<array<ll, 2>> matching;
  for (ll i = 1; i <= n; ++i)
    if (i < lk[i])
      weight += e[i][lk[i]].w,
          matching.push_back({i - 1, lk[i] - 1});
  return {weight, matching};
}
}  // namespace blossom
```

4.9 Floyd-Warshall Algorithm

```
int dis[N][N];
void init(int n) {
  for (int i = 0; i <= n; i++) {  // 初始化
    for (int j = 0; j <= n; j++)
      dis[i][j] = INF;
    dis[i][i] = 0;
  }
}
void floyd_warshall(int n) {
  for (int k = 0; k < n; k++) {  // 窮舉中繼點 k
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {  // 窮舉點對 (i, j)
        dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
      }
    }
  }
}
```

4.10 Maximum Clique

```
#define N 111       // 80 ~ 100 左右
struct MaxClique {  // 0-base
  typedef bitset<N> Int;
  Int linkto[N], v[N];
  int n;
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; i++) {
      linkto[i].reset();
      v[i].reset();
    }
  }
  void addEdge(int a, int b) { v[a][b] = v[b][a] = 1; }
  int popcount(const Int& val) { return val.count(); }
  int lowbit(const Int& val) { return val._Find_first(); }
  int ans, stk[N];
  int id[N], di[N], deg[N];
  Int cans;
  void maxclique(int elem_num, Int candi) {
    if (elem_num > ans) {
      ans = elem_num;
      cans.reset();
      for (int i = 0; i < elem_num; i++)
        cans[id[stk[i]]] = 1;
    }
    int potential = elem_num + popcount(candi);
    if (potential <= ans) return;
    int pivot = lowbit(candi);
    Int smaller_candi = candi & (~linkto[pivot]);
    while (smaller_candi.count() && potential > ans) {
      int next = lowbit(smaller_candi);
      candi[next] = !candi[next];
      smaller_candi[next] = !smaller_candi[next];
      potential--;
      if (next == pivot ||
          (smaller_candi & linkto[next]).count()) {
        stk[elem_num] = next;
        maxclique(elem_num + 1, candi & linkto[next]);
      }
    }
  }
  int solve() {
    for (int i = 0; i < n; i++) {
      id[i] = i;
      deg[i] = v[i].count();
    }
    sort(id, id + n, [&](int id1, int id2) {
      return deg[id1] > deg[id2];
    });
    for (int i = 0; i < n; i++)
      di[id[i]] = i;
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        if (v[i][j]) linkto[di[i]][di[j]] = 1;
    Int cand;
    cand.reset();
    for (int i = 0; i < n; i++)
      cand[i] = 1;
    ans = 1;
    cans.reset();
    cans[0] = 1;
    maxclique(0, cand);
    return ans;
  }
} solver;
```

4.11 Kosaraju's Algorithm for SCC

```
struct Scc {  // 新的 DAG 可能會有多重邊
#define PB push_back
  int n, nScc, vst[MXN], bln[MXN];
  vector<int> E[MXN], rE[MXN], vec, dag[MXN];
  set<int> vis[MXN];
  void init(int _n) {
    n = _n;
    for (int i = 0; i <= n; i++) {
      E[i].clear(), rE[i].clear();
      dag[i].clear(), vis[i].clear();
      bln[i] = -1;
    }
  }
  void addEdge(int u, int v) {
    E[u].PB(v);
    rE[v].PB(u);
  }
  void DFS(int u) {
    vst[u] = 1;
    for (auto v : E[u])
      if (!vst[v]) DFS(v);
    vec.PB(u);
  }
  void rDFS(int u) {
    vst[u] = 1;
    bln[u] = nScc;  // 編號是 0-base
    for (auto v : rE[u])
      if (!vst[v]) rDFS(v);
  }
  void build_new_dag() {
    for (int i = 0; i < n; i++) {
      for (int &j : E[i]) {
        if (bln[i] != bln[j] &&
            vis[bln[i]].count(bln[j]) == 0) {
          dag[bln[i]].push_back(bln[j]);
          vis[bln[i]].insert(bln[j]);
        }
      }
    }
  }
  void solve() {
    nScc = 0;
    vec.clear();
    fill(vst, vst + n + 1, 0);
    for (int i = 0; i < n; i++)
      if (!vst[i]) DFS(i);
```

```
    reverse(vec.begin(), vec.end());
    fill(vst, vst + n + 1, 0);
    for (auto v : vec) {
      if (!vst[v]) {
        rDFS(v);
        nScc++;
      }
    }
    build_new_dag();
  }
};
```

4.12 Kruskal's Algorithm for MST

```
void kruskal() {
  sort(graph, graph + m);  // 將邊照大小排序
  int ans = 0;

  for (int i = 0; i < m; i++) {  //>:)
    if (find(graph[i].u) != find(graph[i].v)) {
      // 如果兩點未聯通
      merge(graph[i].u, graph[i].v);
      // 將兩點設成同一個集合
      ans += graph[i].w;  // 權重加進答案
      if (sz[find(graph[i].u)] == n)
        break;  // 當並查集大小等價於樹內點的數量
    }
  }
}
```

4.13 Kuhn-Munkre Algorithm

```
struct KM {  // max weight, for min negate the weights
  int n, mx[MXN], my[MXN], pa[MXN];
  ll g[MXN][MXN], lx[MXN], ly[MXN], sy[MXN];
  bool vx[MXN], vy[MXN];
  void init(int _n) {  // 1-based
    n = _n;            // 初始化
    for (int i = 1; i <= n; i++)
      fill(g[i], g[i] + n + 1, 0);
  }
  void addEdge(int x, int y, ll w) {  // 加邊
    g[x][y] = w;
  }
  void augment(int y) {
    for (int x, z; y; y = z)
      x = pa[y], z = mx[x], my[y] = x, mx[x] = y;
  }
  void bfs(int st) {
    for (int i = 1; i <= n; ++i)
      sy[i] = INF, vx[i] = vy[i] = 0;
    queue<int> q;
    q.push(st);
    for (;;) {
      while (q.size()) {
        int x = q.front();
        q.pop();
        vx[x] = 1;
        for (int y = 1; y <= n; ++y)
          if (!vy[y]) {
            ll t = lx[x] + ly[y] - g[x][y];
            if (t == 0) {
              pa[y] = x;
              if (!my[y]) {
                augment(y);
                return;
              }
              vy[y] = 1, q.push(my[y]);
            } else if (sy[y] > t) pa[y] = x, sy[y] = t;
          }
      }
      ll cut = INF;
      for (int y = 1; y <= n; ++y)
        if (!vy[y] && cut > sy[y]) cut = sy[y];
      for (int j = 1; j <= n; ++j) {
        if (vx[j]) lx[j] -= cut;
        if (vy[j]) ly[j] += cut;
        else sy[j] -= cut;
      }
      for (int y = 1; y <= n; ++y)
        if (!vy[y] && sy[y] == 0) {
          if (!my[y]) {
            augment(y);
            return;
          }
          vy[y] = 1, q.push(my[y]);
        }
    }
  }
  ll solve() {  // 解決
    fill(mx, mx + n + 1, 0);
    fill(my, my + n + 1, 0);
    fill(ly, ly + n + 1, 0);
    fill(lx, lx + n + 1, -INF);
    for (int x = 1; x <= n; ++x)
      for (int y = 1; y <= n; ++y)
        lx[x] = max(lx[x], g[x][y]);
    for (int x = 1; x <= n; ++x)
      bfs(x);
    ll ans = 0;
    for (int y = 1; y <= n; ++y)
      ans += g[my[y]][y];  // 答案存在 my[]
    return ans;
  }
} graph;  // 找二分圖最大權完美匹配
```

4.14 Max-flow-min-cost

```
struct zkwflow {
  static const int maxN = 100000;
  struct Edge {
    int v, f, re;
    ll w;
  };
  int n, s, t, ptr[maxN];
  bool vis[maxN];
  ll dis[maxN];
  vector<Edge> E[maxN];
  void init(int _n, int _s, int _t) {
    n = _n, s = _s, t = _t;
    for (int i = 0; i < n; i++)
      E[i].clear();
  }
  void add_edge(int u, int v, int f, ll w) {
    E[u].push_back({v, f, (int)E[v].size(), w});
    E[v].push_back({u, 0, (int)E[u].size() - 1, -w});
  }
  bool SPFA() {
    fill_n(dis, n, LLONG_MAX);
    fill_n(vis, n, false);
    queue<int> q;
    q.push(s);
    dis[s] = 0;
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      vis[u] = false;
      for (auto &it : E[u]) {
        if (it.f > 0 && dis[it.v] > dis[u] + it.w) {
          dis[it.v] = dis[u] + it.w;
          if (!vis[it.v]) {
            vis[it.v] = true;
            q.push(it.v);
          }
        }
      }
    }
    return dis[t] != LLONG_MAX;
  }
  int DFS(int u, int nf) {
    if (u == t) return nf;
    int res = 0;
    vis[u] = true;
    for (int &i = ptr[u]; i < (int)E[u].size(); i++) {
      auto &it = E[u][i];
      if (it.f > 0 && dis[it.v] == dis[u] + it.w &&
          !vis[it.v]) {
        int tf = DFS(it.v, min(nf, it.f));
        res += tf, nf -= tf, it.f -= tf;
        E[it.v][it.re].f += tf;
        if (nf == 0) {
          vis[u] = false;
          break;
        }
      }
    }
    return res;
```

```
  }
  pair<int, ll> flow() {
    int flow = 0;
    ll cost = 0;
    while (SPFA()) {
      fill_n(ptr, n, 0);
      int f = DFS(s, INT_MAX);
      flow += f;
      cost += dis[t] * f;
    }
    return {flow, cost};
  }  // reset: do nothing
} flow;
```

4.15 Kuhn's Algorithm

```
vector<int> g[MXN];
bool vis[MXN];
int S[MXN];  // S 為紀錄這個點與誰匹配
int n, ans;  // n: 左集合數量, ans: 紀錄答案

bool dfs(int u) {  // 找最大匹配
  for (int &v : g[u]) {
    if (!vis[v]) {
      vis[v] = true;
      if (S[v] == -1 || dfs(S[v])) {
        S[v] = u;
        return true;
      }
    }
  }
  return false;
}

void solve() {  // 記得每次使用需清空 vis
  memset(S, -1, sizeof(S));
  ans = 0;
  for (int i = 1; i <= n; i++) {  // 跑左集合
    memset(vis, 0, sizeof(vis));
    if (dfs(i)) ans++;
  }
}
```

4.16 Flow Methods

```
【二分圖定理】(M = Max Matching)
  • Min Vertex Cover = M
  • Max Independent Set = |V| - M
  • Min Edge Cover = |V| - M
注意 : 對偶提醒：Min Cut = Max Flow，遇到 Min/Max 轉換卡關
    時，寫出 LP 標準型轉 Dual。
【找二分圖 MVC & MIS 成員】
  跑完 Dinic 後，在殘留網路上從 Source 走「Cap > 0」的邊進
      行 DFS/BFS：
  • MVC = (左側未 visit 的點) U (右側有 visit 的點)
  • MIS = (左側有 visit 的點) U (右側未 visit 的點) (即
      MVC 的補集)
【最大密度子圖】 密度 D = (ΣW_e + ΣW_v) / |V'|
  對答案 D 進行二分搜。取極大常數 S = Σ(所有權重) 或 inf，
      確保邊權非負。
  建圖規則 (Cap)：
  1. S -> 點 v : S
  2. 原圖無向邊 (u, v, w) : 建雙向邊 u -> v 及 v -> u，Cap
       = w
  3. 點 v -> T : S + 2*D - deg[v] - 2*W_v
    (註：deg[v] 為與 v 相連的「邊權重」總和)
  判定與找成員：
  • 若 Max Flow < S * |V|，代表存在密度 > D 的子圖 (D 可繼
      續往上搜)。
  • 找子圖成員：跑完後從 S 走「Cap > 0」的邊，能到達的點集
      即為所求。
【PSP】
  全部可能的收益 - Min Cut = 最大收益
  做完 B 實驗才能做 A 實驗，則 A -> B
```

5 Tree Algorithms

5.1 時間戳記 & 倍增表 (初始化)

```
const int LG_MXN = __lg(MXN) + 2;
vector<int> edge[MXN];  // i 有哪些小孩
int timing = 1, lgN, n;
int in[MXN], out[MXN], anc[MXN][LG_MXN], depth[MXN];
void dfs_init(int u, int f) {  // 0 倍祖先、時間戳記
  in[u] = timing++;            // 這時進入 u
  anc[u][0] = f;               // now 的 0 倍祖先是 f
  for (int v : edge[u]) {
    if (v == f) continue;
    depth[v] = depth[u] + 1;
    dfs_init(v, u);
  }
  out[u] = timing++;  // 這時離開 u
}
void build_table(int n) {  // 初始化
  lgN = __lg(n) + 1;
  dfs_init(1, 0);
  for (int i = 1; i <= lgN; i++)  // 建立倍增表
    for (int now = 1; now <= n; now++)
      anc[now][i] = anc[anc[now][i - 1]][i - 1];
}
```

5.2 LCA

```
bool is_ancestor(int u, int v) {  // 判斷 u 是否 v 是祖先
  return (in[u] < in[v] && out[v] < out[u]);
}
int getLCA(int x, int y) {  // 找最近共同祖先
  if (is_ancestor(x, y))    // 如果 x 為 y 的祖先
    return x;               // 則 LCA 為 y
  if (is_ancestor(y, x))    // 如果 y 為 x 的祖先
    return y;               // LCA 為 y
  for (int i = lgN; i >= 0; i--)  // 逼近 LCA
    if (!is_ancestor(anc[x][i], y) && anc[x][i])
      x = anc[x][i];
  return anc[x][0];  // 回傳此點的父節點即為答案
}
```

5.3 Eular Tour for LCA

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 5;
vector<int> tour, edge[N];
int timing = 0;
int in[N], out[N];
void dfs(int u, int fa) {
  tour.push_back(u);  // 進入的時間點
  in[u] = timing++;
  for (auto v : edge[u]) {
    if (v == fa) continue;
    dfs(v, u);
    tour.push_back(u);  // 走完其中一個孩子 再走回自己
  }
  out[u] = timing++;
}
void build(vector<int>& tour) {  // 把tour丟進線段樹初始化
  seg.build(tour);  // 除了tour，還要建每個點的深度
}
int query(int x, int y) {
  if (is_ancster(x, y)) return x;  // LCA(x,y) = x
  if (is_ancster(y, x)) return y;  // LCA(x,y) = y;
  // 如果「y的區間」在「x的區間」的左邊，就交換
  if (out[y] < in[x]) swap(x, y);
  // 取得區間內哪個位置的深度是最低的
  int index = seg.query(out[x], in[y]);
  return tour[index];
}
```

5.4 樹上差分

```
int tag[N], f[N], ans[N], root;
void add(int x, int y) {
  int lca = getLCA(x, y);
  tag[x]++;
  tag[y]++;
  tag[lca]--;
  if (lca != root) tag[f[lca]]--;
}
int dfs_add(int now, int fa) {
  int diff = tag[now];
  for (auto nxt : edge[now]) {
    if (nxt == fa) continue;
```

```
    diff += dfs_add(nxt, now);
  }
  return ans[now] = diff;
}
```

5.5 DSU on Tree

```
int find(int x) {
  if (f[x] == x) return x;
  else return f[x] = find(f[x]);
}
void merge(int x, int y) {
  x = find(x), y = find(y);
  if (sz[x] < sz[y]) swap(x, y);
  sz[x] += sz[y];
  f[y] = x;
}
void init() {
  // 初始化一開始大小都為1 (每群一開始都是獨立一個)
  for (int i = 1; i <= n; i++) sz[i] = 1;
  for (int i = 1; i <= n; i++) f[i] = i;
}
```

5.6 樹鏈剖分

```
#define REP(i, s, e) for (int i = (s); i <= (e); i++)
#define REPD(i, s, e) for (int i = (s); i >= (e); i--)
const int MAXN = 100010;
const int LOG = 19;
struct HLD {
  int n;
  vector<int> g[MAXN];
  int sz[MAXN], dep[MAXN];
  int ts, tid[MAXN], tdi[MAXN], tl[MAXN], tr[MAXN];
  //  ts : timestamp , useless after yutruli
  //  tid[ u ] : pos. of node u in the seq.
  //  tdi[ i ] : node at pos i of the seq.
  //  tl , tr[ u ] : subtree interval in the seq. of node
  //  u
  int prt[MAXN][LOG], head[MAXN];
  // head[ u ] : head of the chain contains u
  void dfssz(int u, int p) {
    dep[u] = dep[p] + 1;
    prt[u][0] = p;
    sz[u] = 1;
    head[u] = u;
    for (int& v : g[u])
      if (v != p) {
        dep[v] = dep[u] + 1;
        dfssz(v, u);
        sz[u] += sz[v];
      }
  }
  void dfshl(int u) {
    ts++;
    tid[u] = tl[u] = tr[u] = ts;
    tdi[tid[u]] = u;
    sort(ALL(g[u]),
         [&](int a, int b) { return sz[a] > sz[b]; });
    bool flag = 1;
    for (int& v : g[u])
      if (v != prt[u][0]) {
        if (flag) head[v] = head[u], flag = 0;
        dfshl(v);
        tr[u] = tr[v];
      }
  }
  inline int lca(int a, int b) {
    if (dep[a] > dep[b]) swap(a, b);
    int diff = dep[b] - dep[a];
    REPD(k, LOG - 1, 0) if (diff & (1 << k)) {
      b = prt[b][k];
    }
    if (a == b) return a;
    REPD(k, LOG - 1, 0) if (prt[a][k] != prt[b][k]) {
      a = prt[a][k];
      b = prt[b][k];
    }
    return prt[a][0];
  }
  void init(int _n) {
    n = _n;
    REP(i, 1, n) g[i].clear();
  }
  void addEdge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
  }
  void yutruli() {  // build function
    dfssz(1, 0);
    ts = 0;
    dfshl(1);
    REP(k, 1, LOG - 1)
    REP(i, 1, n) prt[i][k] = prt[prt[i][k - 1]][k - 1];
  }
  vector<PII> getPath(int u, int v) {
    vector<PII> res;
    while (tid[u] < tid[head[v]]) {
      res.push_back(PII(tid[head[v]], tid[v]));
      v = prt[head[v]][0];
    }
    res.push_back(PII(tid[u], tid[v]));
    reverse(ALL(res));
    return res;
    /* res : list of intervals from u to v
     * u must be ancestor of v
     * usage :
     * vector< PII >& path = tree.getPath( u , v )
     * for( PII tp : path ) {
     *   int l , r;tie( l , r ) = tp;
     *   upd( l , r );
     *   uu = tree.tdi[ l ] , vv = tree.tdi[ r ];
     *   uu ~> vv is a heavy path on tree
     * }
     */
    int query(int x, int y) {
      int lca = getlca(x, y);
      int ans = 0;

      // x -> lca 點權總和
      while (top[lca] != top[x]) {
        ans += seg.query(dfn[top[x]], dfn[x]);
        x = par[top[x]];
      }
      ans += seg.query(dfn[lca], dfn[x]);

      // y -> lca 點權總和
      while (top[lca] != top[y]) {
        ans += seg.query(dfn[top[y]], dfn[y]);
        y = par[top[y]];
      }
      ans += seg.query(dfn[lca], dfn[y]);

      ans -= seg.query(
          dfn[lca],
          dfn[lca]);  // lca被重複算到一次，要扣掉
    }
  }
} tree;
```

5.7 虛樹

```
vector<int> virTree(vector<int> ver) {
  sort(ver.begin(), ver.end(), cmp);  // 用dfn排序
  vector<int> res(ver.begin(), ver.end());
  for (int i = 1; i < ver.size(); i++) {
    res.push_back(
        lca(ver[i - 1], ver[i]));  // 把LCA丟進虛樹內
  }
  sort(res.begin(), res.end(), cmp);  // 在用dfn排序
  res.erase(
      unique(res.begin(), res.end()),
      res.end());  // 可能會有重複的點，需要去掉重複的
  return res;
}
int count_answer(vector<int> virTree) {
  sort(virTree.begin(), virTree.end(), cmp);
  int ans = 0;
  for (int i = 1; i < virTree.size(); i++) {
    ans += query(lca(virTree[i - 1], virTree[i]),
                 virTree[i]);
  }
  return ans;
}
```

5.8 樹重心

```
int cent;  // 這就是重心
void dfs(int u, int pre) {
  sz[u] = 1;
  int mx = 0;
  for (int& v : g[u]) {
    if (v == pre) continue;
    dfs(v, u);
    sz[u] += sz[v];
    mx = max(mx, sz[v]);
  }
  mx = max(mx, n - sz[u]);
  if (mx <= n / 2) cent = u;
}
```

5.9 重心剖分

```
/* 以上略 */
const int MXN = 2e5 + 5;
vector<int> g[MXN];
int n, k, mx_dep;
ll ans = 0;
bool vis[MXN];
int sz[MXN], cnt[MXN] = {1};

// 計算以 u 為根（忽略已移除的重心與父邊）的子樹大小
int get_sz(int u, int pre) {  // 這個先預處理
  sz[u] = 1;
  for (int& v : g[u]) {
    // 不跨越父親、也不進入已「移除當過重心」的節點
    if (v == pre || vis[v]) continue;
    sz[u] += get_sz(v, u);
  }
  return sz[u];
}

// 在包含 u 的當前子樹內找重心：
// 從 u 出發，若存在子樹大小 >= desired (約 n/2)
// 就往那個子樹走
// 直到找不到更大的子樹為止，回傳當前節點作為重心
int get_cent(int u, int pre, int desired) {
  for (int& v : g[u]) {
    if (v != pre && sz[v] >= desired && !vis[v])
      return get_cent(v, u, desired);
  }
  return u;
}

void cal_cnt(int u, int pre, bool filling, int dep = 1) {
  if (dep > k) return;
  mx_dep = max(
      mx_dep,
      dep);  // 記錄本層最大深度，供稍後清空 cnt[] 使用
  if (filling)
    cnt[dep]++;  // 填表：
                 // 把這個子樹的距離分布，併入「已處理過的
                     子樹」
  else
    ans += cnt[k - dep];  // 查詢：找另一側距離是 (k -
                          // dep) 的節點數，配對成長度 k
  for (int& v : g[u]) {
    if (!vis[v] && pre != v)
      cal_cnt(v, u, filling, dep + 1);
  }
}

void decomp(int u = 1) {
  // 找重心（若 total 偶數，可能落在任一個重心）
  int cent = get_cent(u, 0, get_sz(u, 0) / 2);

  vis[cent] = true;  // 移除重心，切割成多個子樹
  mx_dep = 0;        // 重置本層使用過的最大深度

  // 針對每個鄰接子樹：先查詢、後填表
  // （避免同一子樹內互相配對）
  for (int& v : g[cent]) {
    if (vis[v]) continue;
    cal_cnt(v, cent,
            false);  // 查詢：累加所有「經過重心」的長度=
                     // k 的路徑
    cal_cnt(
        v, cent,
        true);  // 填表：把這個子樹的距離分布併入 cnt[]
  }

  // 重置 cnt[]：保留 cnt[0]=1
  // （代表僅重心），清掉本層曾經動用過的深度範圍 註：
  // 此實作選擇在「離開當前重心」時做清理，確保下一層剛進
      來時
  // cnt 狀態就是正確的
  cnt[0] = 1;
  for (int i = 1; i <= mx_dep; i++) cnt[i] = 0;

  // 遞迴處理每個子樹
  // （此時重心已被標記，不會跨越到其他分支）
  for (int& v : g[cent]) {
    if (!vis[v]) decomp(v);
  }
}
```

5.10 Tree Hash

```
map<vector<int>, int> id;
int dfs(int x, int f) {
  vector<int> sub;
  for (int v : g[x]) {
    if (v != f) sub.push_back(dfs(v, x));
  }
  sort(sub.begin(), sub.end());
  if (!id.count(sub)) id[sub] = id.size();
  return id[sub];
}
```

6 String Algorithms

6.1 Suffix Array (Binary Lifting)

```
struct SA {
    int n;
    vector<int> rk, oldrk, sa, h;
    string s;
    SA(string &t): s(t), n(t.size()), rk(n<<1), oldrk(n
        <<1), sa(n), h(n) {
        for(int i = 0; i < n; i++) rk[i] = s[i], sa[i] = i
            ;

        if (n == 1) {
            rk[0] = 1;
            return;
        }

        for(int w = 1; w < n; w<<=1) {
            sort(sa.begin(), sa.end(), [&](int x, int y){
                return (rk[x]==rk[y]) ? (rk[x+w]<rk[y+w])
                    : (rk[x]<rk[y]);
            });
            oldrk = rk;
            rk[sa[0]] = 1;
            for(int i = 1, p = 1; i < n; i++) {
                if(oldrk[sa[i]]==oldrk[sa[i-1]] && oldrk[
                    sa[i]+w]==oldrk[sa[i-1]+w]) rk[sa[i]]
                    = p;
                else rk[sa[i]] = ++p;
            }
        }

        for(int i = 0, k = 0; i < n; i++) {
            if(rk[i] == 1) continue;
            if(k) k--;
            int j = sa[rk[i]-2];
            while(i+k < n && j+k < n && s[i+k] == s[j+k])
                k++;
            h[rk[i]-1] = k;
        }
    }
  void find(string p) {  // binary search
    int l = 0, r = n, m = p.size();
    if (m > n) return;
    for (int i = 0; i < m; i++) {
      auto cmp = [&](int lhs, int rhs) {
        return s[lhs + i] < rhs;
      };
      auto nl =
          lower_bound(sa + l, sa + r, p[i], cmp) - sa;
      auto nr =
```

```
          lower_bound(sa + l, sa + r, p[i] + 1, cmp) - sa;
      l = nl, r = nr;
      if (l >= r) break;
    }
    cout << (r > l ? "YES" : "NO") << "\n";
  }
};
```

6.2 Suffix Array (SAIS)

```
// 此為後綴陣列模板
// 用法:
// 把要處理的字串轉成整數陣列傳入suffix_array()，會得到SA
    與H維結果
const int N = 500010;  // 有可能要在這就寫原本字串的兩倍長
struct SA {
#define REP(i, n) for (int i = 0; i < int(n); i++)
#define REP1(i, a, b) for (int i = (a); i <= int(b); i++)
  bool _t[N * 2];
  int _s[N * 2], _sa[N * 2], _c[N * 2], x[N], _p[N],
      _q[N * 2], hei[N], r[N];
  int operator[](int i) { return _sa[i]; }
  void build(int* s, int n, int m) {
    memcpy(_s, s, sizeof(int) * n);
    sais(_s, _sa, _p, _q, _t, _c, n, m);
    mkhei(n);
  }
  void mkhei(int n) {
    REP(i, n) r[_sa[i]] = i;
    hei[0] = 0;
    REP(i, n) if (r[i]) {
      int ans = i > 0 ? max(hei[r[i - 1]] - 1, 0) : 0;
      while (_s[i + ans] == _s[_sa[r[i] - 1] + ans])
        ans++;
      hei[r[i]] = ans;
    }
  }
  void sais(int* s, int* sa, int* p, int* q, bool* t,
            int* c, int n, int z) {
    bool uniq = t[n - 1] = true, neq;
    int nn = 0, nmxz = -1, *nsa = sa + n, *ns = s + n,
        lst = -1;
#define MS0(x, n) memset((x), 0, n * sizeof(*(x)))
#define MAGIC(XD)                              \
  MS0(sa, n);                                  \
  memcpy(x, c, sizeof(int) * z);               \
  XD;                                          \
  memcpy(x + 1, c, sizeof(int) * (z - 1));     \
  REP(i, n)                                    \
  if (sa[i] && !t[sa[i] - 1])                  \
    sa[x[s[sa[i] - 1]]++] = sa[i] - 1;         \
  memcpy(x, c, sizeof(int) * z);               \
  for (int i = n - 1; i >= 0; i--)             \
    if (sa[i] && t[sa[i] - 1])                 \
      sa[--x[s[sa[i] - 1]]] = sa[i] - 1;
    MS0(c, z);
    REP(i, n) uniq &= ++c[s[i]] < 2;
    REP(i, z - 1) c[i + 1] += c[i];
    if (uniq) {
      REP(i, n) sa[--c[s[i]]] = i;
      return;
    }
    for (int i = n - 2; i >= 0; i--)
      t[i] =
          (s[i] == s[i + 1] ? t[i + 1] : s[i] < s[i + 1]);
    MAGIC(REP1(i, 1, n - 1) if (t[i] && !t[i - 1])
              sa[--x[s[i]]] = p[q[i] = nn++] = i);
    REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i] - 1]) {
      neq = lst < 0 || memcmp(s + sa[i], s + lst,
                              (p[q[sa[i]] + 1] - sa[i]) *
                                  sizeof(int));
      ns[q[lst = sa[i]]] = nmxz += neq;
    }
    sais(ns, nsa, p + nn, q + n, t + n, c + z, nn,
         nmxz + 1);
    MAGIC(for (int i = nn - 1; i >= 0; i--)
              sa[--x[s[p[nsa[i]]]]] = p[nsa[i]]);
  }
} sa;
int H[N], SA[N];  // SA: 後綴陣列的length
                  // H[i]: SA[i]與SA[i -1]的共同子字串長
void suffix_array(int* ip, int len) {
  // should padding a zero in the back
  // ip is int array, len is array length
  // ip[0..n-1] != 0, and ip[len] = 0
  ip[len++] = 0;
  sa.build(ip, len, 128);
  for (int i = 0; i < len; i++) {
    H[i] = sa.hei[i + 1];
    SA[i] = sa._sa[i + 1];
  }
  // resulting height, sa array \in [0,len)
}
```

6.3 AC Automata

```
struct ACautomata {
  struct Node {
    int cnt, i;
    Node *go[26], *fail, *dic;
    Node() {
      cnt = 0;
      fail = 0;
      dic = 0;
      i = 0;
      memset(go, 0, sizeof(go));
    }
  } pool[1048576], *root;
  int nMem, n_pattern;
  Node* new_Node() {
    pool[nMem] = Node();
    return &pool[nMem++];
  }
  void init() {
    nMem = 0;
    root = new_Node();
    n_pattern = 0;
    add("");
  }
  void add(const string& str) { insert(root, str, 0); }
  void insert(Node* cur, const string& str, int pos) {
    for (int i = pos; i < str.size(); i++) {
      if (!cur->go[str[i] - 'a'])
        cur->go[str[i] - 'a'] = new_Node();
      cur = cur->go[str[i] - 'a'];
    }
    cur->cnt++;
    cur->i = n_pattern++;
  }
  void make_fail() {
    queue<Node*> que;
    que.push(root);
    while (!que.empty()) {
      Node* fr = que.front();
      que.pop();
      for (int i = 0; i < 26; i++) {
        if (fr->go[i]) {
          Node* ptr = fr->fail;
          while (ptr && !ptr->go[i]) ptr = ptr->fail;
          fr->go[i]->fail = ptr =
              (ptr ? ptr->go[i] : root);
          fr->go[i]->dic = (ptr->cnt ? ptr : ptr->dic);
          que.push(fr->go[i]);
        }
      }
    }
  }
  void query(string s) {
    Node* cur = root;
    for (int i = 0; i < (int)s.size(); i++) {
      while (cur && !cur->go[s[i] - 'a']) cur = cur->fail;
      cur = (cur ? cur->go[s[i] - 'a'] : root);
      if (cur->i >= 0) ans[cur->i]++;
      for (Node* tmp = cur->dic; tmp; tmp = tmp->dic)
        ans[tmp->i]++;
    }
  }  // ans[i] : number of occurrence of pattern i
} AC;
```

6.4 Suffix Automata

```
const int MXN = 200005;
struct SAM {
    struct State {
        int len, link, sz;
        int next[26];
    } st[MXN << 1];
```

```
    int sz, last;
    int cnt[MXN << 1], idx[MXN << 1];
    SAM() { init();}
    void init() {
        st[0].len = 0;
        st[0].link = -1;
        st[0].sz = 0;
        memset(st[0].next, -1, sizeof(st[0].next));
        sz = 1;
        last = 0;
    }
    void push(const string& s) { for (char c : s) add(c -
        'a');}
    void add(int ch) {
        int cur = sz++;
        st[cur].len = st[last].len + 1;
        st[cur].sz = 1;
        memset(st[cur].next, -1, sizeof(st[cur].next));
        int p = last;
        while (p != -1 && st[p].next[ch] == -1) {
            st[p].next[ch] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[ch];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = sz++;
                st[clone].len = st[p].len + 1;
                st[clone].link = st[q].link;
                st[clone].sz = 0;
                memcpy(st[clone].next, st[q].next, sizeof(
                    st[q].next));

                while (p != -1 && st[p].next[ch] == q) {
                    st[p].next[ch] = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
    void topo() { // counting sort + bottom-up sz update
        int max_len = 0;
        for (int i = 0; i < sz; i++) {
            max_len = max(max_len, st[i].len);
            cnt[i] = 0;
        }
        for (int i = 0; i < sz; i++) cnt[st[i].len]++;
        for (int i = 1; i <= max_len; i++) cnt[i] += cnt[i
            - 1];
        for (int i = 0; i < sz; i++) idx[--cnt[st[i].len]]
            = i;
        for (int i = sz - 1; i > 0; i--) {
            int u = idx[i], f = st[u].link;
            if (f != -1){
                // root[f] = SegTree::merge(root[f], root[
                    u], 1, n);
                st[f].sz += st[u].sz;
            }
        }
    }
    void trans(const string& t) {
        int cur = 0, step = 0;
        for (int i = 0; i < t.size(); i++) {
            int ch = t[i] - 'a';
            while (cur != -1 && st[cur].next[ch] == -1) {
                cur = st[cur].link;
                if (cur != -1) step = st[cur].len;
            }
            if (cur == -1) {
                cur = 0;
                step = 0;
            } else {
                cur = st[cur].next[ch];
                step++;
            }
        }
    }
} sam;
```

```
// 維護 endpos 集合
namespace SegTree{
    int ls[MXN * 50], rs[MXN * 50];
    int sum[MXN * 50];
    int tot = 0;
    void update(int &u, int l, int r, int pos){
        if(!u) u = ++tot;
        sum[u]++;
        if(l == r) return;
        int mid = l + ((r - l) >> 1);
        if(pos <= mid) update(ls[u], l, mid, pos);
        else update(rs[u], mid+1, r, pos);
    }
    int merge(int u, int v, int l, int r){
        if(!u || !v) return (u ? u : v);
        int node = ++tot; // node = u if no saving is
            required
        int mid = l + ((r - l) >> 1);
        ls[node] = merge(ls[u], ls[v], l, mid);
        rs[node] = merge(rs[u], rs[v], mid+1, r);
        sum[node] = sum[ls[node]] + sum[rs[node]];
        return node;
    }
    int query(int u, int l, int r, int ql, int qr){
        if(!u || ql > r || qr < l) return 0;
        if(ql <= l && r <= qr) return sum[u];
        int mid = l + ((r - l) >> 1);
        int res = 0;
        if(ql <= mid) res += query(ls[u], l, mid, ql, qr);
        if(qr > mid) res += query(rs[u], mid+1, r, ql, qr)
            ;
        return res;
    }
}
```

6.5 Z-algorithm

```
int z[MAXN];
void Z_value(const string& s) {
  // z[i] = lcp(s[1...],s[i...])
  int i, j, left, right, len = s.size();
  left = right = 0;
  z[0] = len;
  for (i = 1; i < len; i++) {
    j = max(min(z[i - left], right - i), 0);
    for (; i + j < len && s[i + j] == s[j]; j++);
    z[i] = j;
    if (i + z[i] > right) {
      right = i + z[i];
      left = i;
    }
  }
}
```

6.6 Knuth-Morris-Pratt Algorithm

```
vector<int> KMP(string s, string t) {
  s = t + '@' + s;
  int sz = s.size();
  vector<int> pi(sz);
  for (int i = 1; i < sz; i++) {
    int len = pi[i - 1];
    while (len != 0 && s[i] != s[len]) len = pi[len - 1];
    if (s[i] == s[len]) pi[i] = len + 1;
  }
  return pi;
}
```

6.7 Manacher's Algorithm

```
vector<int> d1(n), d2(n);
for (int i = 0, l = 0, r = -1; i < n; i++) {
  int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
  while (0 <= i - k && i + k < n && s[i - k] == s[i + k])
    k++;
  d1[i] = k--;
  if (i + k > r) l = i - k, r = i + k;
}
for (int i = 0, l = 0, r = -1; i < n; i++) {
  int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
  while (0 <= i - k - 1 && i + k < n &&
         s[i - k - 1] == s[i + k])
    k++;
  d2[i] = k--;
```

```
  if (i + k > r) l = i - k - 1, r = i + k;
}
```

6.8 Minimum Rotation

```
int min_rotation(string s) {
  int a = 0, N = s.size();
  s += s;
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
      if (a + j == i || s[a + j] < s[i + j]) {
        i += max(0, j - 1);
        break;
      }
      if (s[a + j] > s[i + j]) {
        a = i;
        break;
      }
    }
  }
  return a;
}
// rotate(begin(s),begin(s)+minRotation(s),end(s))
// 上面註解可以直接讓一個字串做 rotate ，得到字典序最小的
// rotation
```

6.9 Rolling Hash

```
ll P1 = 75577, P2 = 12721, MOD = 998244353;
pii hashs[MAXN], hashs2[MAXN];
string s1, s2;
void build(string s, pii hash) {
  ll val1 = 0, val2 = 0;
  for (int i = 0; i < s.length(); i++) {
    val1 = (val1 * P1 % MOD + s[i]) % MOD;
    val2 = (val2 * P2 % MOD + s[i]) % MOD;
    hash[i] = {val1, val2};
  }
}
ll minus(ll a, ll b) { return (a - b + MOD) % MOD; }
ll cnt_occur() {  // 檢查
  int n = s1.size(), k = s2.size();
  ll cnt = (hashs2[k - 1].x == hashs[k - 1].x &&
            hashs2[k - 1].y == hashs[k - 1].y);
  ll val1 = 0, val2 = 0;
  for (int i = 1, j = k; j < n; j++, i++) {
    val1 = minus(hashs[j].x,
                 hashs[i - 1].x * fpow(P1, j - i + 1));
    val2 = minus(hashs[j].y,
                 hashs[i - 1].y * fpow(P1, j - i + 1));
    if (hashs2[k - 1].x == val1 &&
        hashs2[k - 1].y == val2)
      cnt++;
  }
  return cnt;
}
```

6.10 Another Hash

```
const int M = 1e9 + 7;
const int C = 13331;
struct HashString { // 1-base
  int n;
  vector<ll> pows, sums;
  HashString(string s)
      : n(s.size()), pows(n + 1, 1), sums(n + 1) {
    for (int i = 1; i <= n; i++) {
      pows[i] = pows[i - 1] * C % M;
      sums[i] = (sums[i - 1] * C + s[i - 1]) % M;
    }
  }
  // Returns the hash of the substring (l, r]
  ll hash(int l, int r) {
    ll h = sums[r] - sums[l] * pows[r - l];
    return (h % M + M) % M;
  }
};
```

6.11 字典樹

```
struct Trie{
    struct node{
        node *nxt[26];
        int cnt, sz;
        node():cnt(0),sz(0){
            memset(nxt,0,sizeof(nxt));
        }
    };
    node *root;
    Trie(){ root = new node();}
    void insert(const string& s){
        node *now = root;
        for(auto i:s){
            now->sz++;
            if(now->nxt[i-'a'] == NULL){
                now->nxt[i-'a'] = new node();
            }
            now = now->nxt[i-'a'];
        }
        now->cnt++;
        now->sz++;
    }
} trie;
```

6.12 回文樹

```
// len[s]是對應的回文長度
// num[s]是有幾個回文後綴
// cnt[s]是這個回文子字串在整個字串中的出現次數
// fail[s]是他長度次長的回文後綴，aba的fail是a
// fail[s] -> s 建邊是顆樹
const int MXN = 1000010;
struct PalT{
  int nxt[MXN][26],fail[MXN],len[MXN];
  int tot,lst,n,state[MXN],cnt[MXN],num[MXN];
  int diff[MXN],sfail[MXN],fac[MXN],dp[MXN];
  char s[MXN]={-1};
  int newNode(int l,int f){
    len[tot]=l,fail[tot]=f,cnt[tot]=num[tot]=0;
    memset(nxt[tot],0,sizeof(nxt[tot]));
    diff[tot]=(l>0?l-len[f]:0);
    sfail[tot]=(l>0&&diff[tot]==diff[f]?sfail[f]:f);
    return tot++;
  }
  int getfail(int x){
    while(s[n-len[x]-1]!=s[n]) x=fail[x];
    return x;
  }
  int getmin(int v){
    dp[v]=fac[n-len[sfail[v]]-diff[v]];
    if(diff[v]==diff[fail[v]])
        dp[v]=min(dp[v],dp[fail[v]]);
    return dp[v]+1;
  }
  int push(){
    int c=s[n]-'a',np=getfail(lst);
    if(!(lst=nxt[np][c])){
      lst=newNode(len[np]+2,nxt[getfail(fail[np])][c]);
      nxt[np][c]=lst; num[lst]=num[fail[lst]]+1;
    }
    fac[n]=n;
    for(int v=lst;len[v]>0;v=sfail[v])
        fac[n]=min(fac[n],getmin(v));
    return ++cnt[lst],lst;
  }
  void init(const char *_s,int LEN){
    tot=lst=n=0;
    newNode(0,1),newNode(-1,1);
    for(; n<LEN && _s[n];) s[n+1]=_s[n],++n,state[n-1]=
        push();
    for(int i=tot-1;i>1;i--) cnt[fail[i]]+=cnt[i];
  }
}palt;
```

7 Geometry

7.1 Point

```
using ld = long double;
template<class T>
struct pt{
  T x,y;
  pt(T _x,T _y):x(_x),y(_y){}
  pt():x(0),y(0){}

  pt operator * (T  c){ return pt(x*c,y*c);}
  pt operator / (T  c){ return pt(x/c,y/c);}
```

```
__pt operator + (pt a){ return pt(x+a.x,y+a.y);}
__pt operator - (pt a){ return pt(x-a.x,y-a.y);}
__T  operator * (pt a){ return x*a.x + y*a.y;}
__T  operator ^ (pt a){ return x*a.x - y*a.x;}

__auto operator<=>(pt o) const { return (x != o.x) ? x <=>
     o.x : y <=> o.y; } // c++20
__bool operator < (pt a) const { return x < a.x || (x == a
    .x && y < a.y);};
__bool operator== (pt a) const { return x == a.x and y ==
    a.y;};
__friend T ori(pt a, pt b, pt c) { return (b - a) ^ (c - a
    ); }
__friend T abs2(pt a) { return a * a; }
};
// using numbers::pi; // c++20
const ld pi = acos(-1);
const ld eps = 1e-8L;
using Pt = pt<ld>;
int sgn(ld x) { return (x > -eps) - (x < eps); } // dcmp
    == sgn
ld abs(Pt a) { return sqrt(abs2(a)); }
ld arg(Pt x) { return atan2(x.y, x.x); }
bool argcmp(Pt a, Pt b) { // arg(a) < arg(b)
    int f = (Pt{a.y, -a.x} > Pt{} ? 1 : -1) * (a != Pt{});
    int g = (Pt{b.y, -b.x} > Pt{} ? 1 : -1) * (b != Pt{});
    return f == g ? (a ^ b) > 0 : f < g;
}
Pt unit(Pt x) { return x / abs(x); }
Pt rotate(Pt u) { // pi / 2
    return {-u.y, u.x};
}
Pt rotate(Pt u, ld a) {
    Pt v{sin(a), cos(a)};
    return {u ^ v, u * v};
}

istream &operator>>(istream &s, Pt &a) { return s >> a.x
    >> a.y; }
ostream &operator<<(ostream &s, Pt &a) { return s << "("
    << a.x << ", " << a.y << ")";}

bool collinearity(Pt a, Pt b, Pt c) {  // 三點共線
    return ((b - a) ^ (c - a)) == 0;
}
```

7.2 Line

```
struct Line {
    Pt a, b;
    Pt dir() { return b - a; }
};
int PtSide(Pt p, Line L) {
    return sgn(ori(L.a, L.b, p));
}
bool PtOnSeg(Pt p, Line L) {
    return PtSide(p, L) == 0 and sgn((p - L.a) * (p - L.b)
        ) <= 0;
}
Pt proj(Pt p, Line l) {
    Pt dir = unit(l.b - l.a);
    return l.a + dir * (dir * (p - l.a));
}
```

7.3 Circle

```
struct Cir {
    Pt o;
    ld r;
};
bool disjunct(const Cir &a, const Cir &b) { //  分開 || 外
    切 -> 1
    return sgn(abs(a.o - b.o) - a.r - b.r) >= 0;
}
bool contain(const Cir &a, const Cir &b) {
    return sgn(a.r - b.r - abs(a.o - b.o)) >= 0;
}
```

7.4 點線距

```
ld PtSegDist(Pt p, Line l) {
    ld ans = min(abs(p - l.a), abs(p - l.b));
    if (sgn(abs(l.a - l.b)) == 0)  return ans;
    if (sgn((l.a - l.b) * (p - l.b)) < 0) return ans;
    if (sgn((l.b - l.a) * (p - l.a)) < 0) return ans;
    return min(ans, abs(ori(p, l.a, l.b)) / abs(l.a - l.b)
        );
}
ld SegDist(Line l, Line m) {
    return PtSegDist({0, 0}, {l.a - m.a, l.b - m.b});
}
```

7.5 PointInPolygon

```
int inPoly(Pt p, const vector<Pt> &P) {
    const int n = P.size();
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        Pt a = P[i], b = P[(i + 1) % n];
        if (PtOnSeg(p, {a, b})) return 1; // on edge
        if ((sgn(a.y - p.y) == 1) ^ (sgn(b.y - p.y) == 1))
            cnt += sgn(ori(a, b, p));
    }
    return cnt == 0 ? 0 : 2; // out, in
}
```

7.6 Intersection Of Line

```
bool isInter(Line l, Line m) {
    if (PtOnSeg(m.a, l) or PtOnSeg(m.b, l) or
        PtOnSeg(l.a, m) or PtOnSeg(l.b, m))
        return true;
    return PtSide(m.a, l) * PtSide(m.b, l) < 0 and
           PtSide(l.a, m) * PtSide(l.b, m) < 0;
}
Pt LineInter(Line l, Line m) {
    double s = ori(m.a, m.b, l.a), t = ori(m.a, m.b, l.b);
    return (l.b * s - l.a * t) / (s - t);
}bool isInter(Line l, Line m) {
    if (PtOnSeg(m.a, l) or PtOnSeg(m.b, l) or
        PtOnSeg(l.a, m) or PtOnSeg(l.b, m))
        return true;
    return PtSide(m.a, l) * PtSide(m.b, l) < 0 and
           PtSide(l.a, m) * PtSide(l.b, m) < 0;
}
Pt LineInter(Line l, Line m) {
    double s = ori(m.a, m.b, l.a), t = ori(m.a, m.b, l.b);
    return (l.b * s - l.a * t) / (s - t);
}
```

7.7 Intersection Of Circle

```
vector<Pt> CircleInter(Cir a, Cir b) {
    double d2 = abs2(a.o - b.o), d = sqrt(d2);
    if (d < max(a.r, b.r) - min(a.r, b.r) || d > a.r + b.r
        ) return {};
    Pt u = (a.o + b.o) / 2 + (a.o - b.o) * ((b.r * b.r - a
        .r * a.r) / (2 * d2));
    double A = sqrt((a.r + b.r + d) * (a.r - b.r + d) * (a
        .r + b.r - d) * (-a.r + b.r + d));
    Pt v = rotate(b.o - a.o) * A / (2 * d2);
    if (sgn(v.x) == 0 and sgn(v.y) == 0) return {u};
    return {u - v, u + v}; // counter clockwise of a
}
```

7.8 Intersection Of Circle and Line

```
vector<Pt> CircleLineInter(Cir c, Line l) {
    Pt H = proj(c.o, l);
    Pt dir = unit(l.b - l.a);
    double h = abs(H - c.o);
    if (sgn(h - c.r) > 0) return {};
    double d = sqrt(max((double)0., c.r * c.r - h * h));
    if (sgn(d) == 0) return {H};
    return {H - dir *d, H + dir * d};
    // Counterclockwise
}
```

7.9 Convex Hull

```
vector<Pt> Hull(vector<Pt> P) {
    sort(all(P));
    P.erase(unique(all(P)), P.end());
    P.insert(P.end(), P.rbegin() + 1, P.rend());
    vector<Pt> stk;
    for (auto p : P) {
        auto it = stk.rbegin();
        while (stk.rend() - it >= 2 and \
```

```
            ori(*next(it), *it, p) <= 0 and \
            (*next(it) < *it) == (*it < p)) {
            it++;
        }
        stk.resize(stk.rend() - it);
        stk.push_back(p);
    }
    stk.pop_back();
    return stk;
}
```

7.10 旋轉卡尺

```
auto RotatingCalipers(const vector<Pt> &hull) { // 最遠點對 回傳距離平方
    int n = hull.size();
    auto ret = abs2(hull[0]);
    ret = 0;
    if (hull.size() <= 2) return abs2(hull[0] - hull[1]);
    for (int i = 0, j = 2; i < n; i++) {
        Pt a = hull[i], b = hull[(i + 1) % n];
        while(ori(hull[j], a, b) <
             (ori(hull[(j + 1) % n], a, b)))
            j = (j + 1) % n;
        chmax(ret, abs2(a - hull[j]));
        chmax(ret, abs2(b - hull[j]));
    }
    return ret;
}
```

7.11 Convex Hull Trick

```
struct Convex {
    int n;
    vector<Pt> A, V, L, U;
    Convex(const vector<Pt> &_A) : A(_A), n(_A.size()) {
        // n >= 3
        auto it = max_element(all(A));
        L.assign(A.begin(), it + 1);
        U.assign(it, A.end()), U.push_back(A[0]);
        for (int i = 0; i < n; i++) {
            V.push_back(A[(i + 1) % n] - A[i]);
        }
    }
    int inside(Pt p, const vector<Pt> &h, auto f) {
        auto it = lower_bound(all(h), p, f);
        if (it == h.end()) return 0;
        if (it == h.begin()) return p == *it;
        return 1 - sgn(ori(*prev(it), p, *it));
    }
    // 0: out, 1: on, 2: in
    int inside(Pt p) {
        return min(inside(p, L, less{}), inside(p, U,
            greater{}));
    }
    static bool cmp(Pt a, Pt b) { return sgn(a ^ b) > 0; }
    // A[i] is a far/closer tangent point
    int tangent(Pt v, bool close = true) {
        assert(v != Pt{});
        auto l = V.begin(), r = V.begin() + L.size() - 1;
        if (v < Pt{}) l = r, r = V.end();
        if (close) return (lower_bound(l, r, v, cmp) - V.
            begin()) % n;
        return (upper_bound(l, r, v, cmp) - V.begin()) % n
            ;
    }
    // closer tangent point array[0] -> array[1] 順時針
    array<int, 2> tangent2(Pt p) {
        array<int, 2> t{-1, -1};
        if (inside(p) == 2) return t;
        if (auto it = lower_bound(all(L), p); it != L.end
            () and p == *it) {
            int s = it - L.begin();
            return {(s + 1) % n, (s - 1 + n) % n};
        }
        if (auto it = lower_bound(all(U), p, greater{});
            it != U.end() and p == *it) {
            int s = it - U.begin() + L.size() - 1;
            return {(s + 1) % n, (s - 1 + n) % n};
        }
        for (int i = 0; i != t[0]; i = tangent((A[t[0] = i
            ] - p), 0));
        for (int i = 0; i != t[1]; i = tangent((p - A[t[1]
             = i]), 1));
        return t;
    }
    int find(int l, int r, Line L) {
        if (r < l) r += n;
        int s = PtSide(A[l % n], L);
        return *ranges::partition_point(views::iota(l, r),
            [&](int m) {
                return PtSide(A[m % n], L) == s;
            }) - 1;
    };
    // Line A_x A_x+1 interset with L
    vector<int> intersect(Line L) {
        int l = tangent(L.a - L.b), r = tangent(L.b - L.a)
            ;
        if (PtSide(A[l], L) * PtSide(A[r], L) >= 0) return
             {};
        return {find(l, r, L) % n, find(r, l, L) % n};
    }
};
```

7.12 Area Of Circle and Polygon

```
double CirclePoly(Cir C, const vector<Pt> &P) {
    auto arg = [&](Pt p, Pt q) { return atan2(p ^ q, p * q
        ); };
    double r2 = C.r * C.r / 2;
    auto tri = [&](Pt p, Pt q) {
        Pt d = q - p;
        auto a = (d * p) / abs2(d), b = (abs2(p) - C.r * C
            .r)/ abs2(d);
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)), t = min(1., -a +
             sqrt(det));
        if (t < 0 or 1 <= s) return arg(p, q) * r2;
        Pt u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + (u ^ v) / 2 + arg(v, q) *
            r2;
    };
    double sum = 0.0;
    for (int i = 0; i < P.size(); i++)
    sum += tri(P[i] - C.o, P[(i + 1) % P.size()] - C.o);
    return sum;
}
```

7.13 Area Of Circle and Triangle

```
double CircleTriangle(Pt a, Pt b, double r) {
    if (sgn(abs(a) - r) <= 0 and sgn(abs(b) - r) <= 0) {
        return abs(a ^ b) / 2;
    }
    if (abs(a) > abs(b)) swap(a, b);
    auto I = CircleLineInter({{}, r}, {a, b});
    erase_if(I, [&](Pt x) { return !PtOnSeg(x, {a, b}); })
        ;
    if (I.size() == 1) return abs(a ^ I[0]) / 2 +
        SectorArea(I[0], b, r);
    if (I.size() == 2) {
        return SectorArea(a, I[0], r) + SectorArea(I[1], b
            , r) + abs(I[0] ^ I[1]) / 2;
    }
    return SectorArea(a, b, r);
}
```

7.14 Area Of Sector

$\angle AOB * r^2/2$

```
double Sector(Pt a, Pt b, double r) {
  double theta = atan2(a.y, a.x) - atan2(b.y, b.x);
  while (theta <= 0) theta += 2 * pi;
  while (theta >= 2 * pi) theta -= 2 * pi;
  theta = min(theta, 2 * pi - theta);
  return r * r * theta / 2;
}
```

7.15 切線

```
vector<Line> CircleTangent(Cir c, Pt p) {
    vector<Line> z;
    double d = abs(p - c.o);
    if (sgn(d - c.r) == 0) {
        Pt i = rotate(p - c.o);
        z.push_back({p, p + i});
    } else if (d > c.r) {
```

```
        double o = acos(c.r / d);
        Pt i = unit(p - c.o);
        Pt j = rotate(i, o) * c.r;
        Pt k = rotate(i, -o) * c.r;
        z.push_back({c.o + j, p});
        z.push_back({c.o + k, p});
    }
    return z;
}
```

7.16 公切線

```
vector<Line> CircleTangent(Cir c1, Cir c2, int sign1) {
    // sign1 = 1 for outer tang, -1 for inter tang
    vector<Line> ret;
    ld d_sq = abs2(c1.o - c2.o);
    if (sgn(d_sq) == 0) return ret;
    ld d = sqrt(d_sq);
    Pt v = (c2.o - c1.o) / d;
    ld c = (c1.r - sign1 * c2.r) / d;
    if (c * c > 1) return ret;
    ld h = sqrt(max(0.0, 1.0 - c * c));
    for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
        Pt n = Pt(v.x * c - sign2 * h * v.y, v.y * c +
            sign2 * h * v.x);
        Pt p1 = c1.o + n * c1.r;
        Pt p2 = c2.o + n * (c2.r * sign1);
        if (sgn(p1.x - p2.x) == 0 && sgn(p1.y - p2.y) ==
            0)
            p2 = p1 + rotate(c2.o - c1.o);
        ret.push_back({p1, p2});
    }
  return ret;
}
```

7.17 半平面交

```
bool cover(Line L, Line P, Line Q) {
    // PtSide(LineInter(P, Q), L) <= 0 or P, Q parallel
    i128 u = (Q.a - P.a) ^ Q.dir();
    i128 v = P.dir() ^ Q.dir();
    i128 x = P.dir().x * u + (P.a - L.a).x * v;
    i128 y = P.dir().y * u + (P.a - L.a).y * v;
    return sgn(x * L.dir().y - y * L.dir().x) * sgn(v) >=
        0;
}
vector<Line> HPI(vector<Line> P) {
    // Line P.a -> P.b 的逆時針是半平面
    sort(all(P), [&](Line l, Line m) {
        if (argcmp(l.dir(), m.dir())) return true;
        if (argcmp(m.dir(), l.dir())) return false;
        return ori(m.a, m.b, l.a) > 0;
    });
    int n = P.size(), l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        if (i and !argcmp(P[i - 1].dir(), P[i].dir()))
            continue;
        while (l < r and cover(P[i], P[r - 1], P[r])) r--;
        while (l < r and cover(P[i], P[l], P[l + 1])) l++;
        P[++r] = P[i];
    }
    while (l < r and cover(P[l], P[r - 1], P[r])) r--;
    while (l < r and cover(P[r], P[l], P[l + 1])) l++;
    if (r - l <= 1 or !argcmp(P[l].dir(), P[r].dir()))
        return {}; // empty
    if (cover(P[l + 1], P[l], P[r]))
        return {}; // infinity
    return vector(P.begin() + l, P.begin() + r + 1);
}
```

7.18 最小包覆圓

```
Pt Center(Pt a, Pt b, Pt c) {
    Pt x = (a + b) / 2;
    Pt y = (b + c) / 2;
    return LineInter({x, x + rotate(b - a)}, {y, y +
        rotate(c - b)});
}
Cir MEC(vector<Pt> P) {
    mt19937 rng(time(0));
    shuffle(all(P), rng);
    Cir C = {P[0], 0.0};
    for (int i = 0; i < P.size(); i++) {
        if (C.inside(P[i])) continue;
        C = {P[i], 0};
        for (int j = 0; j < i; j++) {
            if (C.inside(P[j])) continue;
            C = {(P[i] + P[j]) / 2, abs(P[i] - P[j]) / 2};
            for (int k = 0; k < j; k++) {
                if (C.inside(P[k])) continue;
                C.o = Center(P[i], P[j], P[k]);
                C.r = abs(C.o - P[i]);
            }
        }
    }
    return C;
}
```

7.19 圓聯集面積

```
// Area[i] : area covered by at least i circle
// TODO:!!!aaa!!!
vector<double> CircleUnion(const vector<Cir> &C) {
    const int n = C.size();
    vector<double> Area(n + 1);
    auto check = [&](int i, int j) {
        if (!contain(C[i], C[j]))
            return false;
        return sgn(C[i].r - C[j].r) > 0 or (sgn(C[i].r - C
            [j].r) == 0 and i < j);
    };
    struct Teve {
        double ang; int add; Pt p;
        bool operator<(const Teve &b) { return ang < b.ang
            ; }
    };
    auto ang = [&](Pt p) { return atan2(p.y, p.x); };
    for (int i = 0; i < n; i++) {
        int cov = 1;
        vector<Teve> event;
        for (int j = 0; j < n; j++) if (i != j) {
            if (check(j, i)) cov++;
            else if (!check(i, j) and !disjunct(C[i], C[j
                ])) {
                auto I = CircleInter(C[i], C[j]);
                assert(I.size() == 2);
                double a1 = ang(I[0] - C[i].o), a2 = ang(I
                    [1] - C[i].o);
                event.push_back({a1, 1, I[0]});
                event.push_back({a2, -1, I[1]});
                if (a1 > a2) cov++;
            }
        }
        if (event.empty()) {
            Area[cov] += pi * C[i].r * C[i].r;
            continue;
        }
        sort(all(event));
        event.push_back(event[0]);
        for (int j = 0; j + 1 < event.size(); j++) {
            cov += event[j].add;
            Area[cov] += (event[j].p ^ event[j + 1].p) /
                2.;
            double theta = event[j + 1].ang - event[j].ang
                ;
            if (theta < 0) theta += 2 * pi;
            Area[cov] += (theta - sin(theta)) * C[i].r * C
                [i].r / 2.;
        }
    }
    return Area;
}
```

7.20 多邊形聯集面積

```
// Area[i] : area covered by at least i polygon
vector<double> PolyUnion(const vector<vector<Pt>> &P) {
    const int n = P.size();
    vector<double> Area(n + 1);
    vector<Line> Ls;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < P[i].size(); j++)
            Ls.push_back({P[i][j], P[i][(j + 1) % P[i].
                size()]});
    auto cmp = [&](Line &l, Line &r) {
        Pt u = l.b - l.a, v = r.b - r.a;
        if (argcmp(u, v)) return true;
        if (argcmp(v, u)) return false;
```

```
        return PtSide(l.a, r) < 0;
    };
    sort(all(Ls), cmp);
    for (int l = 0, r = 0; l < Ls.size(); l = r) {
        while (r < Ls.size() and !cmp(Ls[l], Ls[r])) r++;
        Line L = Ls[l];
        vector<pair<Pt, int>> event;
        for (auto [c, d] : Ls) {
            if (sgn((L.a - L.b) ^ (c - d)) != 0) {
                int s1 = PtSide(c, L) == 1;
                int s2 = PtSide(d, L) == 1;
                if (s1 ^ s2) event.emplace_back(LineInter(
                    L, {c, d}), s1 ? 1 : -1);
            } else if (PtSide(c, L) == 0 and sgn((L.a - L.
                b) * (c - d)) > 0) {
                event.emplace_back(c, 2);
                event.emplace_back(d, -2);
            }
        }
        sort(all(event), [&](auto i, auto j) {
            return (L.a - i.ff) * (L.a - L.b) < (L.a - j.
                ff) * (L.a - L.b);
        });
        int cov = 0, tag = 0;
        Pt lst{0, 0};
        for (auto [p, s] : event) {
            if (cov >= tag) {
                Area[cov] += lst ^ p;
                Area[cov - tag] -= lst ^ p;
            }
            if (abs(s) == 1) cov += s;
            else tag += s / 2;
            lst = p;
        }
    }
    for (int i = n - 1; i >= 0; i--) Area[i] += Area[i +
        1];
    for (int i = 1; i <= n; i++) Area[i] /= 2;
    return Area;
};
```

7.21 Pick's Theory

$$A = i + \frac{b}{2} - 1, \quad 2A = 2i + b - 2, \qquad i = A - \frac{b}{2} + 1$$

其中 A 為面積、b 為圖形內部的點數量、i 為圖形線上的點數量

```
vector<Pt> p;
ll compute_area(int n) {  // A * 2
  ll area = 0;
  for (int i = 1; i < n; i++)
    area += cross(p[i], p[(i + 1) % n], p[0]);
  area = abs(area);
  return area;
}
ll compute_pt_on_line(int n) {  // b
  ll pt = 0;
  for (int i = 0; i < n; i++)
    pt += __gcd(abs(p[i].x - p[(i + 1) % n].x),
                abs(p[i].y - p[(i + 1) % n].y));
  return pt;  // 這是網路上找的
}
ll compute_inner_pt(int n) {  // i
  return (compute_area(n) - compute_pt_on_line(n)) / 2 +
         1;
}
```

8 DP

8.1 區間 DP

時間複雜度 $O(n^3)$，常見的 n 在 500 以內，至多 1,000。

```
// 區間大小為 1 為 base case
for (int i = 1; i <= n; i++) dp[i][i] = 0;
// 由小區間往大開始轉移
for (int len = 2; len <= n; len++)
  for (int l = 1, r = len; r <= n; l++, r++) {
    dp[l][r] = INF;
    for (int k = l; k < r; k++)
      dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] +
                                  n[i] * m[k] * n[r]);
  }
return dp[1][n];
```

8.2 位數 DP

dp[位數][狀態]，dp[pos][state]:
定義為目前位數在前導狀態為 state 的時候的計數
例如：求數字沒有出現 66 的數量 l ~ r
→ dp[pos][1] 可表示計算 pos 個位數在前導出現一個 6 的計數
→ dp[3][1] 則計算 6XXX
模板的 pos 是反過來的，但不影響 (只是用來 dp 記憶用)

```
  pos:    目前位數
  state:  前導狀態
  lead:   是否有前導 0
          (大部分題目不用但有些數字 e.g. 00146 如果有影響時要考慮)
  limit:  使否窮舉有被 num 限制
```

```
vector<int> num;
int dp[20][state];
int dfs(int pos, int state, bool lead, bool limit) {
  if (pos == num.size()) {  // 有時要根據不同state回傳情況
    return 1;
  }
  if (limit == false && lead == false &&
      dp[pos][state] != -1)
    return dp[pos][state];
  int up = limit ? num[pos] : 9;
  int ans = 0;
  for (int i = 0; i <= up; i++) {
    // 有時要考慮那些狀況要continue
    ans += dfs(pos + 1, state || (check[i] == 2),
               lead && i == 0, limit && i == num[pos]);
  }
  if (limit == false && lead == false)
    dp[pos][state] = ans;
  return ans;
}
```

8.3 SOS DP

```
for (int mask = 0; mask < (1 << N); mask++)
  F[mask] = A[mask];
for (int i = 0; i < N; i++)
  for (int mask = 0; mask < (1 << N); mask++)
    if (mask & (1 << i)) F[mask] += F[mask ^ (1 << i)];
```

8.4 凸包優化 DP

```
struct Line {
  mutable ll m, b, p; // when x<=p, this line is the best
  bool operator<(const Line& o) const { return m < o.m; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) { x->p = inf; return false; }
    if (x->m == y->m) x->p = x->b > y->b ? inf : -inf;
    else x->p = div(y->b - x->b, x->m - y->m);
    return x->p >= y->p;
  }
  void insert_line(ll m, ll b) {
    auto z = insert({m, b, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(
        y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll eval(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.m * x + l.b;
  }
};
```

8.5 分治優化 DP

```
// 使用條件：不依賴同層資訊+決策點有單調性+成本函數與分治不衝突
void solve(int k, int l=1, int r=n, int vl=1, int vr=n){
    if(l > r) return;
    int mid = (l + r) >> 1, p = 0;
    dp[k][mid] = 1e18;
    for(int i = vl; i <= min(vr, mid - 1); i ++){
```

```
        if(dp[k][mid] >= dp[k - 1][i] + f(i,j)){ //窮舉最佳轉移點
            dp[k][mid] = dp[k - 1][i] + f(i,j);
            p = i;
        }
    }
    solve(k, l, mid - 1, vl, p); // dp[i][1~mid-1] 的轉移點會在區間 vl~p 之間
    solve(k, mid + 1, r, p, vr); // dp[i][mid+1~r] 的轉移點會在區間 p~vr 之間
}
solve(k, k, n, 1, n);
```

9 Sorting

9.1 Merge Sort

```
template <typename T>
void merge_sort(T &arr, T &tmp, int l, int r) {
  // 陣列元素少於 1 直接結束
  if (r <= l + 1) return;

  // 分成左右兩邊處理
  int m = l + (r - l) / 2;
  merge_sort(arr, tmp, l, m);  // [l, m)
  merge_sort(arr, tmp, m, r);  // [m, r)

  // 合併兩邊的結果
  for (int i = l, li = l, ri = m; i < r; i++) {
    // 只剩下右邊 || 兩邊都有剩，右邊比左邊小
    if (li == m || (ri != r && arr[ri] < arr[li]))
      tmp[i] = arr[ri++];
    // 只剩下左邊 || 兩邊都有剩，左邊比右邊小
    else tmp[i] = arr[li++];
  }

  // 把結果複製回來
  for (int i = l; i < r; i++) arr[i] = tmp[i];
}
```

9.2 Quick Sort

```
template <typename T>
void quick_sort(T arr[], int l, int r) {
    // 陣列元素少於 1 直接結束
    if (r <= l + 1)
        return;

    // 左邊和右邊的指針
    int li = l + 1, ri = r - 1;
    // 判定的基準點
    T pivot = arr[l];

    // TODO: 左邊 <= 基準點 < 右邊
    while (li != ri) {
        // 右邊的指針往左移動，找比基準點小的值
        while (arr[ri] > pivot && li < ri)
            ri--;
        // 左邊的指針往右移動，找比基準點大的值
        while (arr[li] <= pivot && li < ri)
            li++;
        // 當左右指針沒有相遇時，表示不滿足 TODO
        if (li < ri)
            swap(arr[li], arr[ri]);
    }

    // 將基準點移到指針相遇的地方
    arr[l] = arr[li];
    arr[li] = pivot;

    quick_sort(arr, l, li);      // 處理基準點的左邊
    quick_sort(arr, li + 1, r);  // 處理基準點的右邊
}
```

10 Miscellaneous

10.1 Discretization

```
int arr[MXN], tmp[MXN], n;
// arr[i] 是初始陣列, 長度為n, 且 0-base
void discretization() {
    for (int i = 0; i < n; i++)  // 將 arr 複製到 tmp
        tmp[i] = arr[i];
    sort(tmp, tmp + n);                          // 排序
    int len = unique(tmp, tmp + n) - (tmp);  // 把 tmp 裡重複的數字去掉
    for (int i = 0; i < n; i++)
        arr[i] = lower_bound(tmp, tmp + len, arr[i]) - tmp
            ;  // 二分搜
}
```

10.2 Mo's Algorithm

```
struct Query {
  int l, r, id;  // 左界、右界、編號
  Query(int _l, int _r, int _id)
      : l(_l), r(_r), id(_id) {}
  friend bool operator<(const Query &lhs,
                        const Query &rhs) {
    if (lhs.l / k == rhs.l / k)  // 如果在同一塊就比較 r
      return lhs.r < rhs.r;
    return lhs.l < rhs.l;  // 如果在不同快就比較 l
  }
};

vector<Query> q;      // 詢問
void add(int idx) {}  // ... 自己推喔
void sub(int idx) {}  // ... 自己推喔

void mos_algorithm() {
  k = sqrt(n);  // n 個東西分成 k 塊
  sort(q.begin(), q.end());
  for (int i = 0, l = 1, r = 0; i < m; i++) {
    while (l > q[i].l) add(--l);
    while (r < q[i].r) add(++r);
    while (l < q[i].l) sub(l++);
    while (r > q[i].r) sub(r--);
    ans[q[i].id] = cur;  // 每一題長不一樣
  }
}
```

10.3 CDQ 分治

```
// 三維偏序 (求 arr[j] < arr[i] (每一維嚴格小於), i!=j j的個數)
// 先照 x 排序 merge sort排y 最後BIT動態求z的順序個數
// 左區間的 x < 右區間的
void cdq(int ll,int rr){
    if(ll == rr) return;
    int m = (ll+rr)/2;
    cdq(ll,m),cdq(m+1,rr);
    int i = ll,j = m+1,t = 0;
    auto work = [&](){
        ans += BIT.qry(arr[j].z); //計數
        temp[t++] = arr[j++];
    };
    while(i <= m && j <= rr){
        if(arr[i].y <= arr[j].y){
            BIT.add(arr[i].z,1); //二維偏序求法
            temp[t++] = arr[i++];
        }
        else work();
    }
    while(i <= m) temp[t++] = arr[i++];
    while(j <= rr) work();
    BIT.reset(); //操作復原
    rep(k,0,t) arr[k+ll] = temp[k];
}
//[l,r)
auto cdq = [&](auto&& self,auto l,auto r){
    if((r - l) <= 1) return;
    auto m = (r - l) / 2 + l;
    self(self,l,m);
    self(self,m,r);
    auto i = l,j = m;
    auto work = [&](){
        ++j;
    };
    while(i != m && j != r){
        if(arr[*i][1] <= arr[*j][1]) {
            ++i;
        }else work();
    }
```

```
    while(j != r) work();
    clear();
    inplace_merge(l,m,r,[&](auto a,auto b){
        return arr[a][1] < arr[b][1];
    });
};
cdq(cdq,all(ord));//排ord
```

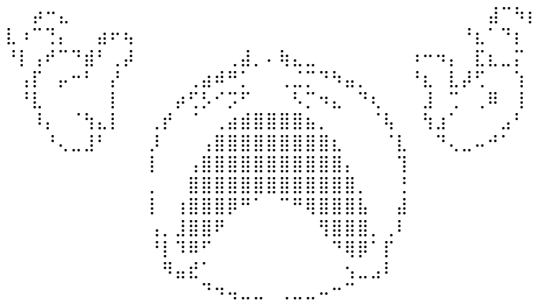
10.4 Decimal

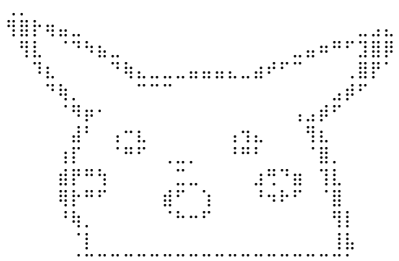
```
from decimal import Decimal, getcontext, ROUND_FLOOR
getcontext().prec = 250 # set precision (MAX_PREC)
getcontext().Emax = 250 # set exponent limit (MAX_EMAX)
getcontext().rounding = ROUND_FLOOR # set round floor
itwo,two,N = Decimal(0.5),Decimal(2),200
pi = angle(Decimal(-1))
```

10.5 Fraction

```
from fractions import Fraction
import math
"""專門用來表示和操作有理數，可以進行算"""
frac1 = Fraction(1)    # 1/1
frac2 = Fraction(1, 3) # 1/3
frac3 = Fraction(0.5)  # 1/2
frac4 = Fraction('22/7') # 22/7
frac5 = Fraction(8, 16) # 自動約分為 1/2
frac9 = Fraction(22, 7)
frac9.numerator   # 22
frac9.denominator # 7
x = Fraction(math.pi)
y2 = x.limit_denominator(100) # 分母限制為 100
print(y2)  # 311/99
float(x) #轉換為浮點數
```

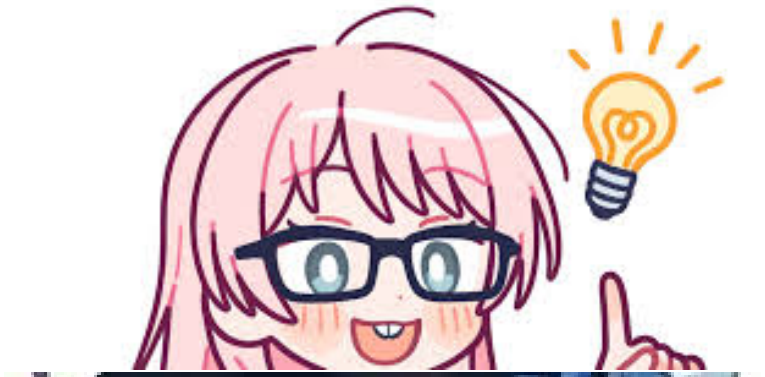
10.6 Blessing





```
                                  @
              (__)    (__) ____/ Hong~Long~Long~Long~
           /| (oo) _  (oo)/----/_____     *
  _o\_____/_|\_\/_/_|__\/|____|//////== *- *  * -
 /__________  \   AC |  AC | NO BUG /== -*
[_____/^^\_____\_____|_____/^^\_____]     *- * -
      \__/             \__/      Chong~Chong~Chong~
```









10.7 Graph Paper